Internship report

Studying structural properties and exact models for solving the Job Sequencing and Tool Switching Problem

Shankar Narayanan

Advisors

Prof. Rafael Colares Prof. Annegret Wagler

Academic supervisor

Dr. Renaud Chicoisne

Laboratoire d'Informatique, de Modélisation et d'Optimisation des Systèmes (LIMOS)

Université Clermont Auvergne

August 26, 2026

Abstract

A machine that makes many different parts needs many different tools, but it can hold only a few of them at a time. Running a set of jobs therefore means stopping to change tools, and each stop costs production time. The Job Sequencing and Tool Switching Problem (SSP) asks for the order of jobs, and the tool loading at each step, that makes the total number of tool changes as small as possible. Choosing the loading for a fixed order is easy. Choosing the order is NP-hard, and it is where all of the difficulty sits.

In practice the SSP is not attacked directly. Jobs are first grouped into as few magazine-fitting batches as possible, and the batches are then sequenced. This two-phase method is fast and is what industry uses, but no worst-case analysis of it has been published. This report gives one. The method admits two readings that the literature has not separated — the cost of a walk through one fixed magazine per group, and the cost the method actually returns once its final step re-optimises the loading — and the two differ on the instances most often used as examples. Within that setting, the optimum is shown to equal the cheapest walk over *all* groupings, so the two-phase gap is exactly the price of insisting on the fewest groups; classes of instances with zero gap are identified for every capacity; upper bounds on the gap are proved for every instance; and the extremal behaviour is worked out at three groups. Positive gaps turn out to be rare, small, and confined to instances whose magazine is large relative to the jobs, which agrees with the one published industrial study. A parameterised cost model places the SSP and the grouping problem at the two ends of a single spectrum, with the transition governed by the gap itself.

On the exact side the obstacle is a single quantity. The strongest compact formulation in the literature has a linear relaxation equal to the number of tools minus the magazine capacity, which is an elementary counting bound. Two methods are built to get past it. A branch-and-Benders-cut algorithm places the ordering in a master problem and settles the loading exactly with a polynomial oracle, so it never forms a weak relaxation. Position-indexed formulations replace the configuration walk by a fixed row set of polynomial size, and one of them, the position–transition formulation, has a linear relaxation that can provably exceed the counting bound.

Both were implemented and benchmarked against three reimplemented compact models on 1,421 instances under one uniform protocol. Whether the counting bound happens to equal the optimum decides the outcome: 94% of the instances where it is tight are closed, against 40% of the rest, and where the solver fails it has usually already found the optimum and cannot certify it. Of four standard strengthenings only the one supplying a better incumbent helps, and the branch-and-price prototypes report the same bound from the other direction by measuring their relaxation value directly.

The cause is structural rather than incidental: the objective counts tool re-insertions, produced by the global interleaving of the schedule, while every inequality either method carries is supported on a single arc or a single position. A family that is not local is derived, proved valid, and shown — by computing what each family could deliver at best — to have roughly four times the remaining headroom of the family the solvers carry today.

Contents

Notation and conventions

Every symbol used in this report is listed here. Each is defined once, in the section given in the last column, and is used with that meaning everywhere else.

Symbol	Meaning	Defined in
n	number of jobs	§2.1
J	the set of jobs, $J = \{1, \ldots, n\}$	§2.1
T	the set of tools	§2.1
$\lvert T \rvert$	number of tools	§2.1
T_j	the tools job j requires	§2.1
b	magazine capacity, in tool slots	§2.1
U	the tools required by at least one job, $U = \bigcup_j T_j$	§2.1
q	the tool excess, $q = \lvert U \rvert - b$	§2.6
π	a job order (a permutation of J)	§2.1
M_i	the magazine contents at position i	§2.1
Z^*	the optimal switch cost of the instance	§2.1
KTNS(π)	the cheapest loading cost for the fixed order π	§2.2
G	a group of jobs	§2.4
K^*	the fewest feasible groups covering all jobs	§2.4
C	a configuration: a set of exactly b tools	§2.3
$\mathcal{C}$	the set of all configurations	§2.3
H_j	the cluster of configurations able to serve job j	§2.3
$d(C, C')$	transition cost, $d(C, C') = \lvert C' \setminus C \rvert$	§2.3
$\mathcal{P}$	a grouping: a partition of J into feasible groups	§2.5
$\gamma(\mathcal{P})$	the cheapest walk through the configurations of $\mathcal{P}$	§4.1
H_{walk}	the walk value of the two-phase heuristic	§2.5
H	the value the two-phase heuristic returns	§2.5
R	re-insertions: insertions of tools evicted earlier	§4.2
R_{opt}	re-insertions made by an optimal schedule	§4.2
w_{ij}	pairwise bound, $w_{ij} = \max(0, \lvert T_i \cup T_j \rvert - b)$	§5.2
ρ	setup cost charged per configuration change	§4.6
θ	the Benders stand-in for the loading cost	§5.2

Two conventions that are fixed once and never varied

Cost. Formulations in the literature charge the first magazine fill differently, and comparing their objective values without adjusting for this produces differences that are not real. Two costs are used here and are always named. The *empty-start* cost counts every insertion, beginning from an empty magazine. The *free-initial* cost does not charge

the first fill. Proposition 5 shows the two differ by a constant that depends only on the instance. Section 2.7 gives the details. Theory is stated in the free-initial cost; the computational study reports the empty-start cost, because that is what the solvers return.

Status of results. Every statement carries a label saying what kind of statement it is.

- **Theorem**, **Proposition**, **Lemma**, **Corollary** — proved, with the proof given here or in Appendix A.
- **Observation** — established by computation, not by proof. The computation is named, and Appendix B says how to repeat it.
- **Conjecture** — supported by computation but not proved. Where a partial argument exists it is given under the heading *Supporting argument*, which is deliberately not the word *Proof*.

Numbers taken from the literature name the paper and the table or section they come from. Numbers computed for this report name the instance family, its parameter range, and the script that produced them.

1 Introduction

1.1 The problem

A workshop has a batch of parts to make. Each part needs its own handful of cutting tools: a drill of one diameter, a particular milling cutter, a tap for a thread. The machine can use any tool the workshop owns, but not all of them at once. Only the tools loaded into its magazine — a rack or carousel with a fixed number of slots — are within reach. Reaching any other one means stopping the machine and swapping a loaded tool out for it.

That swap is where the problem comes from. The machine produces nothing while it happens, and a batch of parts with varied requirements forces a great many of them. The standard model of the situation therefore counts swaps and nothing else, treating cutting times and part handling as fixed because they do not depend on the order the parts are made in. The seven modelling assumptions behind that choice are due to Crama et al. [14] and are set out in Section 2.1.

Two decisions control the count. The first is the order the parts are made in. The second is which tools to keep in the magazine and which to displace at each step. The Job Sequencing and Tool Switching Problem, written SSP throughout this report, makes both decisions together and minimises the total number of tool insertions. Figure 1 shows it at its smallest.

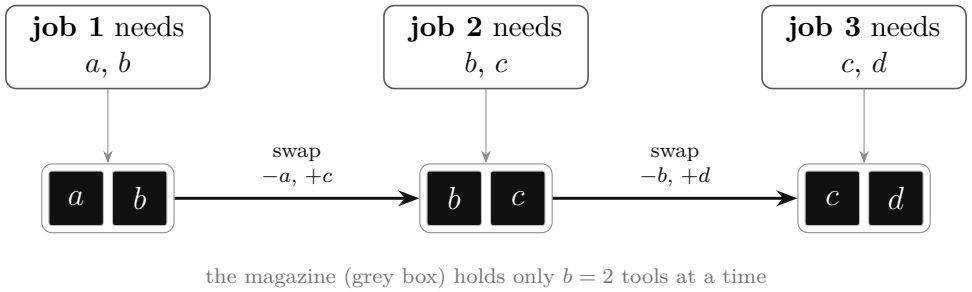

Figure 1: The problem at its smallest. Three jobs run from left to right on a machine whose magazine holds two tools. A job can run only when both of its tools are loaded. Filling the magazine for job 1 costs two insertions. Each later job happens to share one tool with the job before it, so one tool is swapped in at each step: two more insertions, four in total. Changing the order of the jobs, or making the magazine larger, changes that count. Finding the order that minimises the total, over all $n!$ orders, is the SSP.

Figure 2 takes that same instance and changes only the order, which changes the count.

Figure 3 does the same on a real benchmark instance of eight jobs, small enough that all 40,320 orders can be enumerated. It shows how much lies between a good order and a bad one, and where the bound that every exact method starts from sits relative to both.

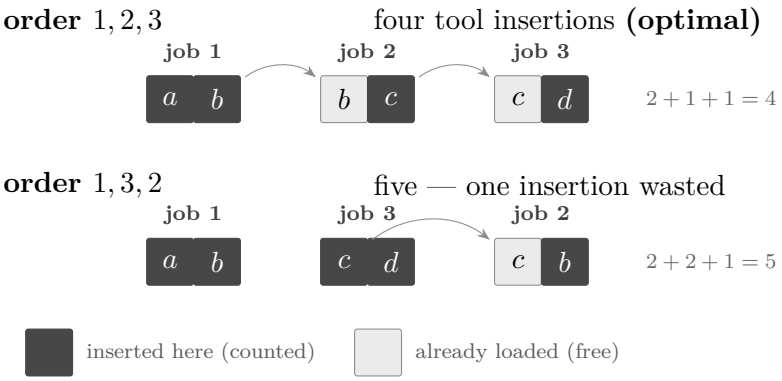

Figure 2: The same three jobs, two orders. Running job 2 between jobs 1 and 3 lets the magazine hand tool b forward and then tool c forward, so only one tool is fetched at each step. Running job 3 second breaks both chains: the magazine must be emptied and refilled, and tool b — discarded after job 1 — has to be fetched a second time. That repeat is the only difference between the two rows, and it is what an exact method has to reason about. There are $n!$ orders and no way to see which chains survive without searching.

The same structure appears well outside machining, and everything in this report carries over. In printed-circuit-board assembly the tools are component feeders and the jobs are the boards. In colour printing they are ink cartridges on a press, and the problem occurs there in exactly the form studied here [7]. In warehouse picking they are the items a picker can carry at once. What matters in each case is that a limited, reusable resource serves groups of demands, and that the order of the demands can be chosen.

1.2 Why the problem is hard, and why it is still open

The two decisions behave very differently.

Loading is easy. Once the order of the jobs is fixed, the cheapest loading policy is known in closed form. It is the Keep Tool Needed Soonest rule of Tang and Denardo [45]: when a tool must be inserted into a full magazine, remove the tool whose next use lies furthest in the future. The rule is optimal, and it runs in $O(n|T|)$ time.

Ordering is hard. Crama et al. [14] proved the SSP NP-hard for every fixed magazine capacity $b \geq 2$. The difficulty is that a good order places jobs with similar tool requirements next to one another, but "similar" here means nothing more than overlapping sets, with no natural distance to measure it, and the magazine limit couples jobs that are far apart in the order.

That combination — an easy inner problem inside a hard outer one — is exactly what decomposition methods are built for, and it is the reason the SSP has attracted exact methods for four decades. Two gaps in that literature motivate this work.

The first is on the practical side. Nobody solves the SSP directly. The standard approach groups the jobs into as few magazine-fitting batches as possible, then orders the

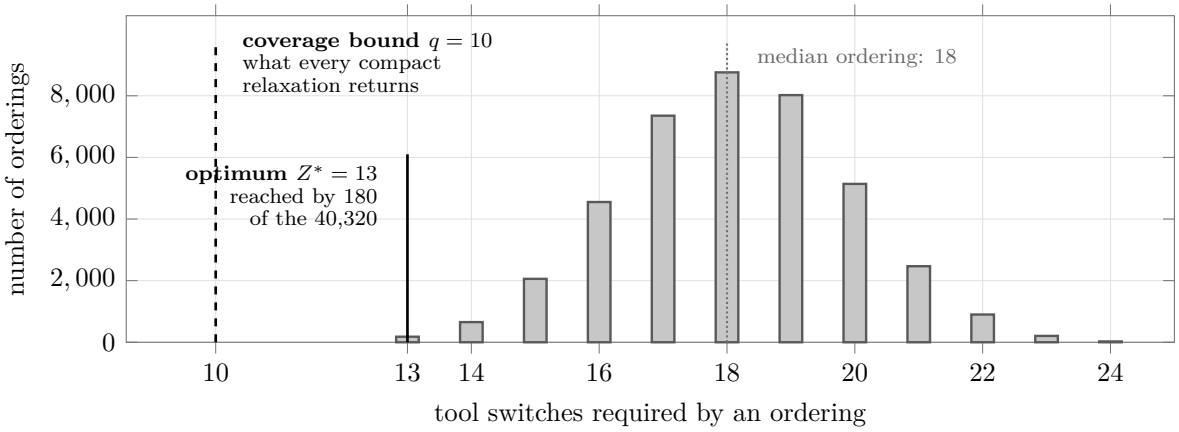

Figure 3: Every one of the 40,320 orderings of instance `L1-1` ($n = 8$ jobs, $|U| = 15$ tools, magazine $b = 5$), enumerated, and the tool switches each requires. Three things are visible at once. A randomly chosen ordering costs 18, and the best costs 13, so the choice of order is worth about a third of the total. Only 180 orderings out of forty thousand are optimal, which is why the outer problem is hard. And the counting bound of $q = 10$, which is what the strongest compact linear relaxation in the literature returns, lies *below the entire distribution*: no ordering whatsoever attains it. A solver given that bound cannot prove any ordering optimal until it has closed a gap of three by branching alone. Section 6 shows what that costs in practice and Section 7 why the bound behaves this way.

batches. It is fast, it is what the strongest metaheuristics start from [32], and where it has been measured against exact optima it does well [7]. No worst-case analysis of it has been published. How far from optimal it can be, and on which instances it is exact, are open questions.

The second is on the exact side. Every compact integer formulation of the SSP proposed so far has a weak linear relaxation, and the strongest of them [16] has a relaxation exactly equal to the number of tools minus the magazine capacity. That is an elementary counting bound: every tool that does not fit in the initial magazine must be inserted at some point. So the strongest relaxation a compact formulation currently offers is this counting argument, and a solver relying on it cannot prove optimality where the bound is loose.

1.3 What was done during this internship

The internship took place at the Laboratoire d'Informatique, de Modélisation et d'Optimisation des Systèmes (LIMOS), under the advisorship of Prof. Rafael Colares and Prof. Annegret Wagler, with Dr. Renaud Chicoisne as academic supervisor. The research directions were set by the advisors, and this report is organised around them. They are recorded here explicitly, so that the origin of each line of work is clear.

- **The configuration-based view of the problem** was proposed by Prof. Colares and Prof. Wagler, and frames the whole project. A *configuration* is a full magazine, and a schedule is a walk through configurations that serves every job. Section 2.3

develops it.

- **The reduction of the SSP to a generalised travelling-salesman problem** was set by Prof. Colares. It is the formal content of that view, and it is what makes both exact methods below natural.
- **The analysis of the gap between the two-phase heuristic and the optimum** was set by Prof. Wagler, together with the request to enumerate the integral solutions of the grouping problem with PORTA. Her specific questions were to construct worst-case examples with the number of groups fixed, to study instances where the gap is attained, and to prove an upper bound tighter than the trivial $b(K^* - 1)$. Section 4 answers the first two and gives several answers to the third.
- **The study of the grouping problem through clutters and blocking duality** was also set by Prof. Wagler. Section 4.5 reports it.
- **The branch-and-Benders-cut algorithm** was proposed by Prof. Colares, following the internship report of Kashou et al. [27], which applies the same architecture to a production-line balancing problem. The suggestion was to try lazy cuts first and cuts at fractional solutions second. Section 5.2 develops it and Section 6.4 measures it.
- **The position-indexed formulations** were proposed by Prof. Colares, including the idea that makes them work: fixing a row set that is polynomial in the instance size, so that column generation only ever adds columns to rows that already exist. The purpose was to make branch-and-price over configurations possible. Section 5.4 develops them.

Work on these directions fell into three parts, and each has a section of its own below.

The first was theoretical: the structural analysis of Section 4. It answers the questions Prof. Wagler set, and raises several that emerged from them.

The second was implementation. The branch-and-Benders-cut solver, the two branch-and-price prototypes, three formulations from the literature reimplemented as baselines, a hybrid genetic heuristic, the campaign harness and the cluster pipeline were built and validated against brute-force optima before any of them was used to produce a result. Section 5 describes the methods and Appendix C the code.

The third was experimental: the benchmark campaign of Section 6, run on the LIMOS cluster over the standard instance families under a uniform protocol, together with the small-scale enumeration used throughout Section 4 to test each statement as it was made. One correction made during the campaign is worth recording, because it changed the results substantially: the Benders master was found to be missing the tool coverage bound, and the campaign was re-run once it was added.

1.4 Conventions

Section 2 defines the problem and everything used to attack it, including a self-contained account of the integer-programming background in Section 2.8, which a reader who already knows integer programming may skip. Section 3 reviews the literature. Longer proofs are in Appendix A, the verification program in Appendix B, and Appendix C says how to reproduce every experiment.

Every statement carries a label — Theorem, Proposition, Lemma, Corollary, Observation, or Conjecture — and the labels are used strictly. A Proposition has a proof. An Observation was established by running a program. A Conjecture has neither, only evidence. Where an argument is given but has not been independently checked it is headed *Supporting argument* rather than *Proof*. The distinction is kept even where it makes the text less confident.

2 The problem and the tools used to attack it

This section defines the SSP, the rule that solves its inner problem, the grouping view that the whole report is built on, and the two elementary lower bounds that everything later is measured against. It ends with a self-contained account of the integer-programming background used in Section 5.

2.1 The Job Sequencing and Tool Switching Problem

An instance consists of n jobs $J = \{1, \dots, n\}$, a set of tools T, a magazine capacity $b \in \mathbb{N}$, and for each job j a set of required tools $T_j \subseteq T$ with $1 \le |T_j| \le b$. The requirement $|T_j| \le b$ is necessary: a job needing more tools than the magazine holds could never be run. Write $U = \bigcup_{j \in J} T_j$ for the set of tools that some job requires. Tools outside U play no part and are ignored.

Definition 1 (Schedule and switch cost)**.** A *schedule* is a pair $(\pi, \mathbf{M})$, where π is a permutation of J giving the order in which the jobs run, and $\mathbf{M} = (M_1, \dots, M_n)$ gives the magazine contents at each position. It is feasible when

$$|M_i| \le b \quad \text{and} \quad T_{\pi(i)} \subseteq M_i \qquad (i = 1, \dots, n),$$

that is, when the magazine never overflows and always holds what the current job needs. With $M_0 = \varnothing$, the *switch cost* of the schedule is the number of tool insertions it makes,

$$\sum_{i=1}^{n} |M_i \setminus M_{i-1}|.$$

The SSP asks for a feasible schedule of minimum switch cost. That minimum is written Z^*.

Counting insertions rather than removals is the standard cost model. Under a fixed capacity each insertion into a full magazine forces exactly one removal, so the two counts differ only at the start of the schedule. Section 2.7 makes that difference precise.

The following instance is used as an example throughout the report.

Example 1 (The 6-ring). The 6-*ring* has six jobs, six tools and capacity $b = 3$. Job j requires $T_j = \{j,\, j+1\}$, with indices taken modulo 6, so that $T_6 = \{6, 1\}$. Consecutive jobs share exactly one tool and non-consecutive jobs share none. Its job–tool incidence matrix, with rows for tools and columns for jobs and a blank denoting 0, is

	1	2	3	4	5	6
t_1	1					1
t_2	1	1				
t_3		1	1			
t_4			1	1		
t_5				1	1	
t_6					1	1

Figure 4 draws it. Its optimum is $Z^* = 6$ in the empty-start cost, or 3 once the first fill is not charged; Section 2.2 computes this.

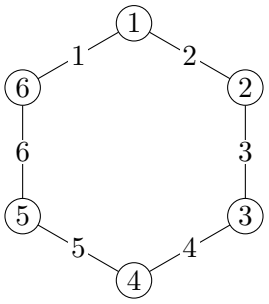

Figure 4: The 6-ring. The six jobs lie on a cycle. Each edge is labelled by the single tool its two endpoint jobs share, so job j requires the two tools on the edges meeting it. Adjacent jobs overlap in one tool; non-adjacent jobs are tool-disjoint.

2.2 Loading tools for a fixed order

Suppose the order of the jobs is already decided. What remains is to choose the magazine contents at each position. This inner problem has a closed-form solution.

Proposition 1 (45)**.** *Fix a permutation π. The following rule produces a magazine sequence of minimum switch cost. Process the jobs in the order π. Whenever a required*

tool is absent, insert it; if the magazine is full, first remove a tool whose next required use lies furthest in the future, removing a tool never required again if one exists. This is the Keep Tool Needed Soonest *(KTNS) rule. It runs in $O(n|T|)$ time, and no loading policy for π costs less.*

The proof is in Tang and Denardo [45]; Crama et al. [14] give a second proof through interval matrices. Write $\mathrm{KTNS}(\pi)$ for the value the rule returns. Since the rule is optimal for every fixed order,

$$Z^* \;=\; \min_{\pi}\; \mathrm{KTNS}(\pi),$$

so the SSP reduces to a pure sequencing problem. Every method in this report is some way of searching over orders while delegating the loading to this rule.

Example 2 (KTNS on the 6-ring). Take the 6-ring and the cyclic order $1, 2, 3, 4, 5, 6$. Figure 5 traces it. The magazine starts empty and holds three tools.

Jobs 1 and 2 need $\{1, 2\}$ and $\{2, 3\}$, so the first magazine is filled with $\{1, 2, 3\}$: three insertions, and this serves both jobs. At position 3, job 3 needs $\{3, 4\}$. Tool 3 is already loaded, so only tool 4 is inserted, and one resident tool must leave. This is where the rule does its work. Of the residents $\{1, 2, 3\}$, tool 2 is never required again, while tool 1 is required by job 6 at the very end. The rule removes tool 2 and keeps tool 1. Positions 4 and 5 repeat the pattern, inserting one new tool and removing the one whose next use is furthest away. Position 6 needs $\{6, 1\}$, and both are present, so it is free.

The total is three insertions to fill the magazine plus one at each of positions $3, 4, 5$: six insertions, or three once the first fill is not charged. Keeping tool 1 loaded through three positions where it is not used is what the rule buys. A policy that removed tool 1 at position 3 would have to reload it at position 6 and would pay seven.

Remark 1 (The loading problem is offline caching). For a fixed order the loading problem is exactly offline paging. The magazine is a cache of size b, position i requests the set $T_{\pi(i)}$, and an insertion is a cache miss. Proposition 1 is then the tool-switching form of Bélády's optimal replacement rule, which removes the item whose next request is furthest in the future and which minimises misses on any fixed request sequence [5]. The comparison is informative. Paging takes the request order as given; the SSP may reorder the jobs, and it is that freedom which turns a polynomially solvable caching problem into an NP-hard one.

2.3 Configurations

The magazine, not the job, is the natural unit for what follows.

Definition 2 (Configuration, cluster, transition cost)**.** A *configuration* is a set $C \subseteq T$ with exactly $|C| = b$ tools. The set of all configurations is written $\mathcal{C}$. For a job j, the

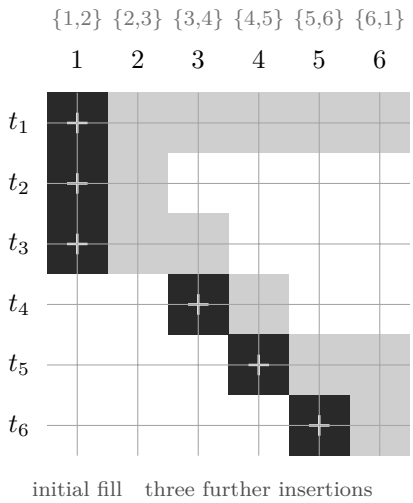

Figure 5: KTNS on the 6-ring in the order $1, 2, 3, 4, 5, 6$. Columns are the six processing positions, with the tools each job needs written above them; rows are the six tools. A shaded cell means the tool sits in the magazine at that position; a "+" marks an insertion. Every column holds exactly three tools. Tool 1, in the top row, is loaded once and kept across all six positions, idle in the middle, so that it need not be reloaded for job 6. The cost is three insertions to fill the magazine and three afterwards: six in the empty-start cost, three in the free-initial cost.

cluster $H_j = \{C \in \mathcal{C} : T_j \subseteq C\}$ is the set of configurations able to serve j. For two configurations, the *transition cost* is the number of tools that must be inserted to pass from one to the other,

$$d(C, C') \;=\; |C' \setminus C|.$$

Restricting attention to full magazines costs nothing. Keeping a tool that might be useful later never increases the number of insertions, so some optimal schedule keeps the magazine full at every position whenever $|U| \geq b$.

Under this view a schedule is a walk $C_{(1)}, \ldots, C_{(r)}$ through $\mathcal{C}$ that visits at least one configuration of every cluster H_j, and its cost is $\sum_l d(C_{(l)}, C_{(l+1)})$. That a schedule and such a walk have equal cost is immediate in one direction: take any schedule, delete repetitions from its sequence of magazines, and what remains is a walk of the same cost that serves every job. The converse is Theorem 1 in Section 4.1.

The SSP is therefore a generalised travelling-salesman problem over $\mathcal{C}$, in which the clusters *overlap*: one configuration can serve many jobs. This formulation is due to Colares and Wagler [12]. The overlap is what separates it from the textbook generalised travelling-salesman problem with disjoint clusters [19, 37, 42], and it is why the standard reductions for that problem do not transfer.

2.4 Grouping the jobs

Section 2.2 settled the loading problem, so what remains is the order. Solving for the order directly is hard, so practice does something indirect. It first asks a simpler question: which jobs could share a magazine?

Definition 3 (Feasible group)**.** A set of jobs $G \subseteq J$ is a *feasible group* if the tools its jobs need, taken together, fit in the magazine:

$$\left| \bigcup_{j \in G} T_j \right| \;\leq\; b.$$

A feasible group can be run from start to finish without a single tool change. Load the tools its jobs need, run them in any order, and only then change anything. This makes the following question natural.

Definition 4 (Job Grouping Problem)**.** The *Job Grouping Problem* (JGP) is the problem of partitioning J into as few feasible groups as possible. The smallest possible number of groups is written K^*.

The JGP ignores order completely. It asks only how the jobs may be divided. It is itself NP-hard, but it is far easier in practice than the SSP, and it is solved exactly on every instance size considered here, by the asymmetric-representatives formulation of Jans and Desrosiers [26] or by the set-covering column generation of Crama and Oerlemans [13].

Two problems now sit next to each other. The JGP says which jobs may share a magazine. The SSP asks for an order. The standard heuristic joins them, in three steps.

1. **Group.** Solve the JGP. This returns K^* feasible groups $G_1, \ldots, G_{K^*}$, and for each group a configuration C_l containing everything that group needs.
2. **Sequence.** Decide the order of the groups. Moving from configuration C to configuration C' costs $d(C, C')$, so finding the cheapest order of the K^* configurations is a travelling-salesman problem on K^* points, which is small enough to solve exactly [2, 23].
3. **Evaluate.** Write out the jobs group by group, in the order chosen in step 2, to obtain a single job order. Apply KTNS to that order and report its cost.

This is the method studied throughout this report. It is fast, it is what practitioners use, and its output is always a feasible schedule, so its cost is never below Z^*.

Example 3 (The heuristic on the 6-ring). Step 1 returns $K^* = 3$ for the 6-ring. One optimal grouping is

$$G_1 = \{1, 2\}, \qquad G_2 = \{3, 4\}, \qquad G_3 = \{5, 6\},$$

whose tool requirements are $\{1,2,3\}$, $\{3,4,5\}$ and $\{5,6,1\}$. Each has exactly three tools, so each group fixes its configuration with no freedom left: $C_1 = \{1,2,3\}$, $C_2 = \{3,4,5\}$, $C_3 = \{5,6,1\}$.

Step 2 compares them. Any two share exactly one tool, so moving between any two costs two insertions. Every order of the three groups costs the same, and step 2 may return any of them. Take G_1, G_2, G_3.

Step 3 writes out the jobs group by group, giving the order $1,2,3,4,5,6$, and applies KTNS. That is the order traced in Example 2, and its cost is six insertions, or three once the first fill is not charged.

Two features of this example matter later.

First, the method does not fully determine its own answer. Step 1 may return several optimal groupings, step 2 may face ties as it does here, and the order of the jobs *inside* a group is left open by all three steps. Different choices can give different costs. Throughout this report the heuristic is read in its most favourable form: its cost is the best value reachable over all of these choices. This is the reading that makes a comparison with Z^* meaningful, because any weaker reading would measure the tie-breaking rule rather than the method.

Second, and less obviously, step 3 does something that step 2 did not anticipate. That is the subject of the next subsection.

2.5 Two ways to measure the heuristic

Steps 1 and 2 reason entirely in terms of configurations. Each group is assigned one magazine, and that magazine is imagined to stay loaded while the whole group runs. Under that picture a schedule is a walk through K^* configurations, and its cost is the sum of the transition costs along the walk.

Step 3 does not do this. It discards the configurations, keeps only the job order they produced, and hands that order to KTNS. KTNS may change the magazine between any two consecutive jobs, including two jobs of the same group. The group structure survives only as an ordering of the jobs; it does not constrain the magazine.

The two readings give different numbers. Both appear in the literature without being distinguished. This report separates them.

Definition 5 (The walk value and the heuristic value)**.** Let $\mathcal{P}$ range over the partitions of J into K^* feasible groups. For each such partition let the configurations be chosen and ordered as cheaply as possible, and write

$$H_{\text{walk}} \;=\; \min_{\mathcal{P}} \; \min_{\sigma} \; \sum_{l=1}^{K^*-1} d\Big(C_{\sigma(l)}, C_{\sigma(l+1)}\Big)$$

for the cheapest such walk. Write H for the cost of the two-phase heuristic as defined in Section 2.4: the minimum, over the same partitions, over all orders of the groups and all orders of the jobs within each group, of the KTNS cost of the resulting job order.

Proposition 2. *For every instance,* $H \leq H_{\text{walk}}$.

Proof. Fix a partition and an order of its groups attaining H_{walk}. Writing the jobs out group by group in that order gives a job order. Holding each group's configuration fixed while its jobs run is one admissible loading for that order, and its cost is exactly the walk cost. By Proposition 1, KTNS returns the cheapest loading for that order, so the KTNS cost is at most the walk cost. Since H minimises over at least this one job order, $H \leq H_{\text{walk}}$. □

The inequality can be strict, and it is strict on the running example.

Example 4 (The two values differ on the 6-ring). Continue Example 3. The walk pays three insertions to build $C_1 = \{1, 2, 3\}$, then two to reach $C_2 = \{3, 4, 5\}$, then two to reach $C_3 = \{5, 6, 1\}$. That is $H_{\text{walk}} = 7$, or 4 free-initial. KTNS on the same job order costs 6, or 3 free-initial. Since $Z^* = 3$ free-initial, the heuristic is *optimal* on the 6-ring while the walk value is one above the optimum.

Figure 6 shows where the two part company. The walk requires the middle group to run under $C_2 = \{3, 4, 5\}$ and nothing else. Tool 1 is not in C_2, so it is removed at position 3 and must be reloaded at position 5 for job 6. KTNS is not bound to C_2. It holds $\{1, 3, 4\}$ at position 3 and $\{1, 4, 5\}$ at position 4, two different magazines inside one group, each serving its own job, and so keeps tool 1 loaded throughout. The one re-insertion is avoided.

Remark 2 (Why the distinction is kept throughout). It would be simpler to work with H_{walk} alone. It has a clean description, a shortest walk through K^* configurations, and the structural results of Section 4 are naturally proved about it.

That simplification is not available, for two reasons. The first is that H_{walk} is not what the method returns. A practitioner running the three steps of Section 2.4 obtains H, so any statement about how far the method can be from optimal must be a statement about H. The second is that the two differ on exactly the instances most often used as examples, as Example 4 shows.

Every result in Section 4 therefore names the value it is about. Proposition 2 carries upper bounds in one direction: anything proved to bound H_{walk} from above also bounds H. Nothing transfers in the other direction. In particular, an instance on which H_{walk} exceeds Z^* is not by itself an instance on which the heuristic is suboptimal.

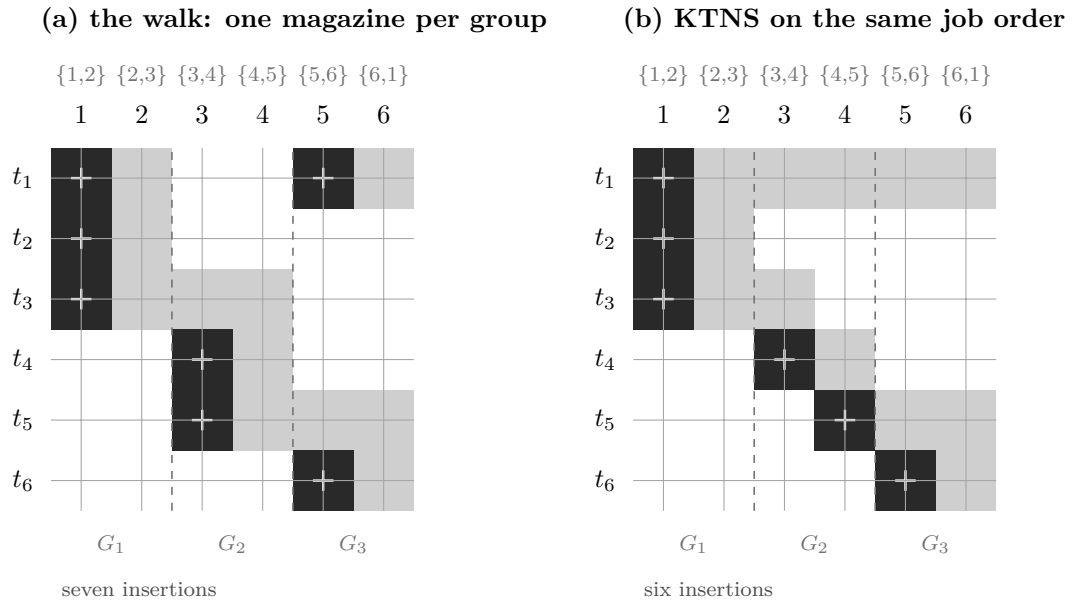

Figure 6: The same grouping of the 6-ring, measured two ways. Columns are the six processing positions, with the tools each job needs above them; rows are the six tools. A shaded cell means the tool sits in the magazine at that position; a "+" marks an insertion. The dashed lines separate the three groups. In (a) each group runs under one fixed magazine, so tool 1, in the top row, is removed at position 3 and reloaded at position 5 for job 6: seven insertions. In (b) KTNS may change the magazine inside a group, so tool 1 is loaded once and kept across all six positions, and the count is six. That single re-insertion is the whole difference between H_{walk} and H.

2.6 Two lower bounds

Two elementary bounds on Z^* recur throughout. Both are stated in the free-initial cost of Section 2.7, which does not charge the first magazine fill. Write $q = |U| - b$ for the number of used tools in excess of the capacity; the quantity appears often enough to deserve a name.

Proposition 3 (Grouping bound)**.** $Z^* \geq K^* - 1$.

Proof. Fix any schedule and list its magazine states. Collapse each run of equal consecutive states to a single representative, giving a sequence $C_{(1)}, \ldots, C_{(r)}$ in which consecutive entries differ. Every job is served by some state, hence by some $C_{(l)}$. Assign each job to one configuration containing its requirement. This partitions J into at most r groups, each with combined requirement inside one configuration and therefore of size at most b. That is a feasible grouping, so $r \geq K^*$. Each of the $r - 1$ transitions inserts at least one tool, so the free-initial cost is at least $r - 1 \geq K^* - 1$. □

Proposition 4 (Coverage bound)**.** $Z^* \geq |U| - b = q$.

Proof. Every tool of U is required by some job, so it occupies the magazine at some position, so it is inserted at least once unless it belongs to the first fill. The first fill holds

at most b tools. Hence at least $|U| - b$ insertions are made beyond the first fill. □

Corollary 1. $Z^* \geq \max\{K^* - 1,\ q\}$.

Neither bound dominates the other, and their maximum is not always attained. On the 6-ring, $K^* - 1 = 2$ while $q = 6 - 3 = 3$, so the coverage bound is tight and the grouping bound is slack. On instances built from many nearly tool-disjoint groups the reverse holds. And both can be slack at once: the eight-job instance with $T_j = \{j, j+1, j+2\}$ modulo 8 and $b = 4$ has $K^* - 1 = 3$ and $q = 4$, but $Z^* = 5$.[1]

The coverage bound q deserves particular attention, because Section 5.1 shows that the strongest compact formulation in the literature has a linear relaxation exactly equal to it. It is therefore both an elementary counting argument and the limit that twenty years of formulation work has reached.

2.7 Cost conventions

Definition 6 (Empty-start and free-initial costs)**.** The *empty-start* cost counts every insertion, starting from an empty magazine as in Definition 1. The *free-initial* cost does not charge the first fill.

Proposition 5 (Convention identity)**.** *For every schedule that keeps the magazine as full as possible,*

$$cost_{\text{empty}} \;=\; cost_{\text{free}} + \min(b, |U|).$$

Proof. An optimal loading keeps the magazine full, so the first position loads $\min(b, |U|)$ tools. The empty-start count charges these; the free-initial count, by definition, does not; and every later insertion is counted identically by both. □

The shift is a constant for a given instance. Three consequences are used later. Differences such as $H - Z^*$ are the same in both conventions. Ratios such as H/Z^* are not, so a ratio is meaningless unless the convention is named. And when objective values produced by different formulations are compared, they must first be brought to a common convention; Section 6.2 describes how the computational study does this.

2.8 Background on integer programming

Section 5 builds two exact methods, and both rest on a small body of standard material. It is set out here, with the examples drawn from the SSP rather than from generic instances. A reader familiar with integer programming may go directly to Section 3.

[1] Verified by exhaustive computation; see Appendix B.

Integer linear programs. An *integer linear program* minimises a linear function of decision variables subject to linear constraints, with some variables required to take whole-number values. In this report those are almost always 0/1 variables encoding a discrete choice: $x_{ij} = 1$ when job j runs immediately after job i, or $g_{t,i} = 1$ when tool t occupies the magazine at position i.

A small example makes the shape concrete. Suppose the order of the jobs is already fixed and only the loading is to be decided. Let $g_{t,i} \in \{0, 1\}$ say whether tool t is in the magazine at position i, and let $s_{t,i} \geq 0$ count an insertion of t at position i. The loading problem is then

$$\begin{aligned} \min \quad & \sum_{t,i} s_{t,i} \\ \text{subject to} \quad & g_{t,i} = 1 && \text{for every } t \in T_{\pi(i)}, \\ & \textstyle\sum_t g_{t,i} \leq b && \text{for every position } i, \\ & s_{t,i} \geq g_{t,i} - g_{t,i-1}, \quad s_{t,i} \geq 0 && \text{for every } t, i, \end{aligned}$$

with $g_{t,0} = 0$. The first family forces the current job's tools to be present, the second is the capacity limit, and the third charges an insertion whenever a tool is in the magazine at position i but was not at position $i - 1$. This model returns to the report as the Benders subproblem in Section 5.2.

The linear relaxation and the integrality gap. Dropping the integrality requirement leaves the *linear relaxation*, an ordinary linear program. Linear programs are solved efficiently, and because the relaxation permits everything the integer program permits and more, its optimum is a *lower bound* on the integer optimum. The difference between the two is the *integrality gap*. A formulation whose gap is small is called *strong*.

Strength is not automatic, and the SSP gives the extreme illustration. Laporte et al. [28] showed that the linear relaxation of the original formulation of Tang and Denardo [45] has optimum 0 on every instance, whatever the true optimum: the fractional solution that spreads every job evenly across every position satisfies all the constraints at zero cost. A bound of 0 is useless, and much of the exact-methods literature reviewed in Section 3 is an attempt to repair this.

Branch and bound. A bound becomes a proof of optimality through search. Branch and bound solves the relaxation; if the answer is integral it is optimal and the search stops. Otherwise some variable takes a fractional value, and the problem is split into two subproblems, one forcing that variable up and one forcing it down. Every integer solution survives in one of the two, so nothing is lost. The subproblems are explored recursively, and a subproblem is discarded as soon as its relaxation bound is no better than the best complete solution already found.

Two quantities therefore govern how long the search takes. The *incumbent* is the best complete solution found so far, and it gives an upper bound. The *dual bound* is the best lower bound over the unexplored subproblems. The search ends when they meet. It matters which of the two is lagging, and Section 7 returns to this, because for the SSP it is almost always the dual bound.

Cutting planes. A *cut* is a linear inequality satisfied by every integer solution but violated by the current fractional one. Adding it removes that fractional point without removing any genuine solution, so the relaxation becomes stronger. Doing this repeatedly before branching, and again at the nodes of the tree, is *branch and cut*. It is what commercial solvers such as CPLEX do internally, and it is the framework into which the Benders algorithm of Section 5.2 is embedded.

A cut may be added as a *lazy constraint*, checked only when a candidate integer solution appears, or as a *user cut*, added at fractional points during the search. The distinction matters for the SSP and is measured in Section 6.4.

Duality, and what dual prices mean. Every linear program has a partner, its *dual*, with the same optimal value. The dual has one variable per constraint of the original, and at an optimal solution that variable is the *price* of the constraint: the rate at which the optimum would change if the constraint were relaxed slightly.

In the loading program above, the dual variable attached to "tool t must be present at position i" measures how much that requirement costs. This is not an abstraction; it is precisely what the Benders algorithm reads off in order to build its cuts, and it is available at no extra cost from the same solve. Duality also supplies a certificate: any feasible solution of the dual gives a valid lower bound on the original problem, whether or not it is optimal.

Total unimodularity. Some integer programs have no integrality gap at all. If the constraint matrix is *totally unimodular*, meaning every square submatrix has determinant 0, 1 or -1, then the vertices of the relaxation are integral, and solving the linear program already solves the integer program. The loading program above has this property: its matrix has the consecutive-ones structure, which is totally unimodular. That is a second proof that the loading problem is easy, and it is the reason the Benders subproblem in Section 5.2 can be solved as a linear program with no loss.

Benders decomposition. Some problems split into a hard part and a part that becomes easy once the hard part is fixed. The SSP is one: choose the order, which is hard, and then load the tools, which is easy. Benders decomposition exploits this.

The *master problem* contains only the hard variables, with the cost of the easy part replaced by a single variable θ that starts optimistically low. Given a candidate solution of the master, the easy *subproblem* is solved exactly. If its true cost exceeds θ, then θ was lying, and one linear inequality is added to the master to say so. The inequality is constructed from the subproblem's dual solution, and it is valid for every master solution, not only the current one. Each such cut removes the current over-optimistic guess without removing any genuine schedule. Repeating drives θ upward until it equals the true cost, at which point the master solution is optimal. Figure 9 in Section 5.2 draws the loop.

Two variants are relevant. In *logic-based* Benders decomposition [25] the subproblem is settled by a combinatorial algorithm rather than by linear-programming duality, and the cut is derived from the algorithm's answer. In *combinatorial* Benders decomposition [11] the cuts are combinatorial statements about which choices can occur together. Both appear in Section 5.2, because KTNS is exactly such a combinatorial oracle.

Column generation and branch-and-price. Column generation runs the same idea in reverse. Some formulations have very few constraints but astronomically many variables, one per configuration, say. Such a model cannot be written out. Instead a *restricted master problem* is kept over a small pool of variables and solved. A separate *pricing problem* then asks whether any variable currently omitted would improve the solution if it were added. The pricing problem searches the full variable space using the current dual prices, and returns the most promising variable, called a *column*. The restricted master is re-solved with it included, and the loop repeats. When the pricing problem reports that no improving column exists, the restricted master has the same optimal value as the full model.

Wrapping this inside branch and bound gives *branch-and-price* [4, 30]. There is one difficulty. Branching on a single column is usually ineffective and forces the pricing problem to carry a growing list of forbidden columns, which changes its structure. A branching rule that leaves the pricing problem structurally unchanged is called *robust*, and Section 5.4 uses one.

3 State of the art

This section reviews the SSP literature. It is organised around one question: how strong a lower bound can be obtained, and by what means. That question organises the exact literature completely, and it also explains why the heuristic literature looks the way it does.

3.1 Origins: the loading rule, and the hardness of ordering

Tang and Denardo [45] posed the problem and settled its inner level. For a fixed job order the optimal loading is the Keep Tool Needed Soonest rule, proved optimal and computable in $O(n|T|)$ time (Proposition 1). Two further elements of that paper are used directly in this report. The first is the pairwise bound: if jobs i and j run consecutively, at least $\max(0, |T_i \cup T_j| - b)$ tools must be switched between them, because the magazine holds b tools but must serve T_i and then T_j. The second is their observation that the shortest Hamiltonian path under those pairwise weights is a lower bound on the optimum. Both reappear in the Benders master of Section 5.2.

Crama et al. [14] made the difficulty precise. They proved the SSP NP-hard for every fixed capacity $b \geq 2$, by showing that finding a minimum-length travelling-salesman path in the edge graph of an arbitrary graph can be written as an SSP instance with $b = 2$, and then extending the construction to larger b by padding the incidence matrix. The same paper fixes the seven modelling assumptions that have been standard in the field since: the magazine is always full, each tool occupies one slot, switch times are equal for all tools, tools are changed one at a time, tool requirements are fixed in advance, tools do not wear out, and the job list is known.

That paper also gives a second, linear-programming proof of KTNS optimality through interval matrices, and its treatment of the non-uniform case is relevant to Section 5.2. When tools have different setup times, the greedy argument breaks: Crama et al. [14] state explicitly that in that setting the greedy algorithm and KTNS are *no longer valid*. What survives is the linear-programming formulation. The matrix remains an interval matrix and therefore totally unimodular, so the loading problem is still solvable in polynomial time, by solving the linear relaxation. This distinction matters here because the Benders subproblem of Section 5.2 is that linear program, not the KTNS rule, and so remains exact in a regime where the rule does not apply.

A later study [15] draws the boundary of tractability sharply. Once tools may occupy several magazine slots, the loading problem alone becomes strongly NP-hard, by reduction from 3-partition, and yet for a fixed capacity it remains polynomial through a shortest-path model. The uniform SSP studied here, whose loading problem is the paging problem of Remark 1 [5], sits at the tractable edge of a family that becomes intractable as soon as the tools stop being uniform.

3.2 The compact formulations, and where they stop

The exact side of the field is a two-decade effort to give the ordering problem a linear relaxation strong enough to drive branch and bound. Its trajectory is the single most important piece of context for this report.

Laporte et al. [28] replaced the original Tang–Denardo model, and it is their paper that established why a replacement was needed: they exhibited a feasible solution of the Tang–Denardo linear relaxation with objective value 0, valid on every instance. Their own formulation, written here as LSS, introduces a dummy job as a start-and-end depot and uses arc variables, magazine-state variables and switch variables on a travelling-salesman skeleton. They strengthen it with three families of valid inequalities, and report that on the Tang–Denardo example the relaxation rises from 0 to 6.0 against an integer optimum of 7.0.

They then solve the model two ways. A linear-programming branch-and-cut, seeded with the GENIUS heuristic of Hertz et al. [24], and a branch-and-bound that carries no linear relaxation at all but two combinatorial bounds on partial sequences, in the enumerative spirit later developed by Yanasse et al. [47]. Their computational tables report the branch-and-cut on instances with eight jobs, and the branch-and-bound reaching twenty-five. That a search with no relaxation beats a search with one is the first concrete evidence that on this problem the bound, not the search, is the limiting factor.

Ghiani et al. [21] recast the SSP as a least-cost Hamiltonian cycle problem with a nonlinear arc cost. They establish a symmetry property: a job sequence and its reversal have the same total switch cost. They are careful to note that this does not make the problem symmetric, since individual transition costs do differ; it is the total that is preserved. The property halves the search space, and they combine it with comb inequalities and lower bounds recomputed inside the search.

Catanzaro et al. [9] produced the tightest compact models to date, a family of five formulations in arc-based variables. Two of their results are used here. Formulations 3 and 4 have the same linear relaxation, and Formulation 4 is the smaller of the two; their computational section says so explicitly and excludes Formulation 3 on that ground. Formulation 5 has a relaxation at least as strong as Formulation 4, and their experiments confirm it is strictly stronger on most of their datasets. Their conclusions recommend Formulation 5 as the best integer-programming formulation for the SSP. The present study uses Formulation 4 as its compact baseline, because that is what was implemented; Formulation 5 was not implemented, and Section 7.4 records this as a limitation rather than a preference.

da Silva et al. [16] give a multicommodity-flow model in which each tool routes one unit of flow through a position network. It requires no lazy constraints, so a standard solver can be used directly, and it includes a symmetry-breaking constraint that eliminates half of the symmetric solutions. Its relaxation is what makes it important. They prove that the linear relaxation of their model equals $M - C$ in their notation: the number of tools minus the magazine capacity. Their argument assumes, as they state, that all M tools are required by some job. Under that assumption $M = |U|$, and the bound is exactly the coverage bound q of Proposition 4. They report solving 17.01% more instances than

Year	Work	Approach	Contribution used here
1988	Tang and Denardo [45]	ILP; the KTNS rule	KTNS optimal for a fixed order; the pairwise bound w_{ij}
1994	Crama et al. [14]	complexity; heuristics	NP-hard for every fixed $b \geq 2$; the seven standard assumptions; KTNS invalid under non-uniform tool costs, the LP is not
2004	Laporte et al. [28]	the LSS model; B&C and B&B	the Tang–Denardo relaxation is identically 0; B&B to 25 jobs beats B&C at 8
2010	Ghiani et al. [21]	Hamiltonian cycle	reversal symmetry preserves the total cost
2015	Catanzaro et al. [9]	compact F1–F5	F3 and F4 share a relaxation, F4 is smaller; F5 is strongest and is their recommendation
2021	da Silva et al. [16]	multicommodity flow	relaxation $= M - C$ when every tool is used; +17.01% instances solved
2024	Akhundov and Ostrowski [1]	symmetry; arc flow	symmetry-broken position and flow models
2025	Legrand et al. [29]	dynamic programming, A^*	combinatorial bounds computed inside the search

Table 1: The exact-method lineage for the SSP. Two decades of compact formulations raised the linear relaxation only as far as the coverage bound $q = |U| - b$ [16]. Moving past it has required leaving the compact linear-programming setting.

the models of Tang and Denardo [45], Laporte et al. [28] and Catanzaro et al. [9], on the instance sets of Yanasse et al. [47].

That is where the compact models stop: the strongest compact relaxation available coincides with a counting argument that fits in one sentence. da Silva et al. [16] also record, for the earlier models, that their relaxations fall below $M - C$ in the great majority of cases; this is a statement about most instances, not all, and Table 3 in Section 5 is hedged accordingly.

3.3 The configuration view

A parallel line abandons job positions for tool configurations.

Crama and Oerlemans [13] formulate the Job Grouping Problem as a set-covering model solved by column generation. Its relaxation is strong, and its pricing subproblem is a Set-Union Knapsack Problem [22], which is NP-hard even under restrictive conditions. That difficulty returns as the bottleneck of the pricing problems in Section 5.4. Jans and Desrosiers [26] give a compact asymmetric-representatives alternative, which is what the grouping solver used in this work implements.

The configuration view that unifies grouping and sequencing is developed by Colares and Wagler [12], work in progress in the advisors' own group. It casts the SSP as a generalised travelling-salesman problem whose clusters overlap, and solves it by a non-compact integer program with exponentially many transition variables, together with generalised subtour-elimination constraints separated as lazy constraints. The overlap is what separates the problem from the textbook generalised travelling-salesman problem with disjoint clusters surveyed by Pop et al. [42], whose Noon–Bean reduction [37] and branch-and-cut methods [19] therefore do not transfer directly. Where a pricing problem generates constraints that depend on the columns already generated, the column-and-row

framework of Muter et al. [36] is the relevant methodology.

Two recent theses sit closest to this work, and both come from the same research group. Otiai [38] builds an exact branch-and-price for the Job Grouping Problem using Ryan–Foster branching [44]. Moreira [35] studies the SSP through the magazine-configuration formulation and measures its relaxation against the compact models. His measurements are directly comparable with those of Section 6, and one of them is quoted there: he reports that the trivial bound $M - b$ obtained by the multicommodity model was the best available bound on all thirty instances of three of his subgroups, while on the tighter subgroup the configuration model gave the best bound on four instances.

3.4 Heuristics and metaheuristics

Because the exact methods scale poorly, most of the SSP literature is heuristic. Early work pairs a constructive travelling-salesman-style rule with local search over the job order [14]. The GENI and GENIUS heuristics of Hertz et al. [24] sharpen those constructions and outperform the earlier ones; GENIUS also supplies the upper bound inside the branch-and-cut of Laporte et al. [28]. Enumerative schemes with dominance pruning [47] and the bounding arguments of Ghiani et al. [21] complete the pre-metaheuristic period.

The current state of the art is the hybrid genetic search of Mecler et al. [32], which couples an order-based genetic algorithm with local search and seeds its population from grouping solutions. The grouping view is therefore built into the best heuristic as well as into the exact methods.

3.5 The decomposition in industrial practice

The group-then-sequence decomposition is not only a theoretical construct. In colour printing the SSP appears in exactly the form studied here, with ink cartridges as tools and the cartridge rack as the magazine. Burger et al. [7] solve the industrial problem by this decomposition, modelling the grouping as a unicost set-covering problem and the sequencing as a travelling-salesman problem, and enumerating optimal groupings by Stirling-number arguments.

Their assessment of the decomposition is the closest thing in the literature to the question Section 4 attacks, and it is conditional. They concur with the earlier finding that the decomposition does well, but qualify it: the agreement holds *especially in cases where the largest job size is only slightly smaller than the magazine size*, and, as the difference between the magazine size and the largest job size grows, the quality of the solutions the heuristic finds decreases. That qualification matches the structural analysis of Section 4 closely, and Section 7.3 returns to the agreement.

Their paper also poses, as an open question in its closing paragraph, the trade-off between minimising the number of tools changed and the number of times tools are

changed, for varying values of the setup time incurred when the machine is stopped. Section 4.6 answers that question for the uniform model.

What the literature does not contain is a worst-case analysis of the decomposition: bounds on its gap or ratio, extremal instances, or conditions for exactness. The survey of Calmels [8] classifies sixty-one SSP papers and identifies gaps in the field; among them is the observation that few authors propose decomposition methods for the SSP at all. No worst-case guarantee has been published for any SSP heuristic, as far as could be determined during this internship.

3.6 Recent exact developments

The exact frontier has moved again recently, by routes that reinforce the reading above. Akhundov and Ostrowski [1] exploit the reversal symmetry of Ghiani et al. [21] systematically, deriving symmetry-broken position-based and arc-flow formulations from a job-group representation. Legrand et al. [29], with Catanzaro among the authors, leave integer programming altogether for a dynamic-programming and A^* search equipped with a new family of lower bounds, reporting the strongest exact results to date.

Their bounds are combinatorial and are computed inside the search rather than as a linear relaxation. That is the pattern. Every recent advance has come from leaving the compact linear-programming setting, whether for a search with sharper combinatorial bounds, for a decomposition whose subproblem is settled exactly, or for a relaxation engineered to exceed the counting bound. The two methods of Section 5 take the second and third of those routes.

3.7 Benders decomposition

The solver of Section 5.2 inherits a developed methodology. Rahmaniani et al. [43] survey the acceleration of Benders decomposition. The particular setting in which the subproblem is settled by a combinatorial oracle rather than by linear-programming duality is the logic-based Benders of Hooker and Ottosson [25] and the combinatorial Benders cuts of Codato and Fischetti [11]. Two classical strengthenings are used later: the Pareto-optimal cut of Magnanti and Wong [31], which selects among alternative dual optima the cut that cuts deepest at a fixed interior point, and the practical form of Papadakos [39], which removes the extra constrained solve the original method required.

The immediate source, however, is not a paper but an internship. Kashou et al. [27] applies branch-and-Benders-cut to a robust production-line balancing problem, integrating Benders into the branch-and-bound tree through lazy-constraint callbacks, replacing a general solver in the dual subproblem with four tailored algorithms, and strengthening the method with a heuristic that pre-injects cuts. He reports that even so the decomposition does not systematically beat the non-decomposed formulation. It was that report which

prompted Prof. Colares to propose the same architecture for the SSP, on the reasoning that the SSP occupies the regime the assembly-line problem lacked: its subproblem is not a general program needing tailored algorithms, but a polynomially solvable oracle, so the cut is exact and essentially free at every callback.

3.8 What is open, and what this report attacks

Three observations set the agenda.

First, the two-phase heuristic has a strong empirical record, one conditional industrial assessment [7], and no worst-case theory. Section 4 builds one.

Second, on the exact side the strongest compact relaxation equals the coverage bound q [16], and the configuration relaxation does not improve on it on most instances [35]. Progress therefore requires either a method that does not rely on a linear relaxation at all, which is the Benders route of Section 5.2, or a formulation whose relaxation can exceed the bound, which is the position–transition route of Section 5.4.

Third, the configuration view that works well for the Job Grouping Problem [38] has no tractable exact counterpart for the SSP, because the natural model carries exponentially many subtour constraints. That is the opening the position-indexed formulations address.

4 How far can the two-phase heuristic be from optimal?

The two-phase heuristic of Section 2.4 is the standard practical method for the SSP. Where it has been measured against exact optima its suboptimality is small [7]. No worst-case analysis of it has been published. This section gives one.

The source of any gap is easy to state. The heuristic commits to a grouping with the *fewest* groups before it sequences anything, whereas the optimal schedule may be served by a larger grouping with cheaper transitions. Everything below makes that statement precise, bounds the resulting loss, and identifies when it is zero.

Which value is being bounded. Section 2.5 distinguished two values: the walk value H_{walk}, and the value H that the method returns. Every result below names the one it is about. The distinction is not a technicality. Section 4.4 shows that the canonical example family of this problem has a positive walk gap and a zero heuristic gap, so a result proved about H_{walk} and reported as a result about the heuristic would be false.

Every statement in this section was additionally verified computationally by the independent program of Appendix B; the labelling convention is the one set out in Section 1.4.

4.1 The gap is the price of using the fewest groups

A grouping is given a cost by ordering its configurations as cheaply as possible.

Definition 7 (Feasible grouping and its cost)**.** A *feasible grouping* is a partition $\mathcal{P} = \{G_1, \ldots, G_p\}$ of J into feasible groups, together with a configuration $C_l \supseteq \bigcup_{j \in G_l} T_j$ for each group. Its cost is the cheapest free-initial walk through its configurations,

$$\gamma(\mathcal{P}) \;=\; \min_{\sigma \in S_p} \sum_{l=1}^{p-1} d\Big(C_{\sigma(l)}, C_{\sigma(l+1)}\Big).$$

The Job Grouping Problem seeks a feasible grouping of minimum cardinality K^*, and H_{walk} is γ minimised over those. Removing the cardinality restriction makes the framework exact.

Theorem 1 (Grouping exactness)**.** $Z^* = \min_{\mathcal{P}} \gamma(\mathcal{P})$*, where the minimum runs over* all *feasible groupings, of every cardinality.*

The proof is in Appendix A. It is a pair of constructions. Any feasible grouping, processed group by group, is a schedule of cost $\gamma(\mathcal{P})$, which gives one inequality. Any optimal schedule, with its jobs grouped by the magazine under which they run, is a feasible grouping of no greater cost, which gives the other.

Corollary 2 (The walk gap)**.**

$$H_{\text{walk}} - Z^* \;=\; \min_{|\mathcal{P}| = K^*} \gamma(\mathcal{P}) \;-\; \min_{\mathcal{P}} \gamma(\mathcal{P}) \;\geq\; 0,$$

and it is strictly positive exactly when no minimum-cardinality grouping attains the overall minimum of γ*.*

This locates the loss exactly: it is the price of the cardinality restriction imposed by the grouping phase, and not a failure of the sequencing phase, which orders optimally whatever it is given.

Example 5 (Where the ring loses, in the walk model)*.* The optimum of the 6-ring is realised by a *four*-configuration grouping,

$$\{1,2,3\} \;\to\; \{1,3,4\} \;\to\; \{1,4,5\} \;\to\; \{1,5,6\},$$

in which consecutive configurations differ in a single tool, for $\gamma = 3$. Every minimum-cardinality grouping uses three configurations and costs 4 (Example 4). The walk gap of one is exactly the difference in Corollary 2: the cheaper grouping is not of minimum cardinality, so the grouping phase never offers it to the sequencer.

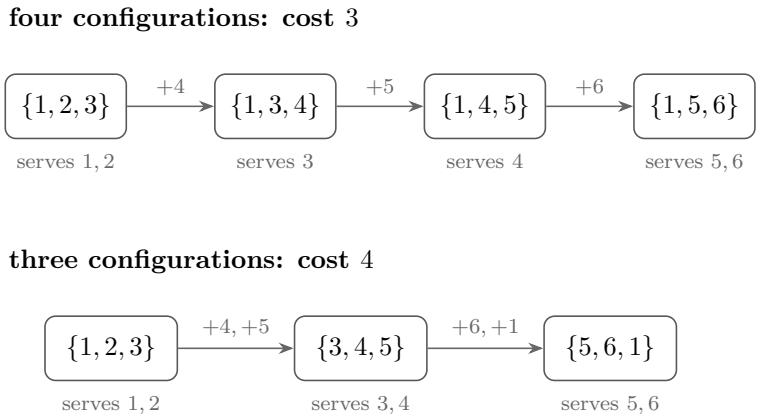

Figure 7: The 6-ring in configuration space, in the walk model. Above: the optimal schedule visits four configurations, each one tool from the next, for a cost of three. Below: the heuristic may use only the $K^* = 3$ minimum-cardinality groups, and every pair of the resulting configurations differs in two tools, so the best it can do in this model is four. The extra unit is one re-insertion of tool 1, which the middle configuration cannot hold. Figure 6 shows what happens when the KTNS step is applied instead.

4.2 Zero-gap classes and worst-case guarantees

This subsection proves bounds that hold for *every* instance and every capacity. Two quantities recur. The first is the tool excess $q = |U| - b$ of Section 2.6. The second is the number of *re-insertions* a schedule makes: insertions of tools that were in the magazine earlier and were removed. Write R_{opt} for the re-insertions made by an optimal schedule. Throughout, $K^* \geq 2$, since at $K^* = 1$ the heuristic is trivially exact, and all costs are free-initial.

Lemma 1 (Cost identity)**.** *Restrict every magazine state to a subset of U, which is possible whenever $K^* \geq 2$ and never increases cost. The free-initial cost of any full-magazine schedule then equals $q + R$, where $R \geq 0$ counts re-insertions.*

Proof. Each of the q tools of U outside the first fill is required by some job, so it is inserted at least once. Split all insertions into first-time insertions, of which there are exactly q, and the rest, which are by definition the R re-insertions. □

Hence $Z^* = q + R_{\text{opt}}$, and writing R_{walk} for the walk model's re-insertions, $H_{\text{walk}} - Z^* = R_{\text{walk}} - R_{\text{opt}}$. The walk gap is exactly the excess re-insertions forced by the cardinality restriction. On the 6-ring the single extra insertion is one such re-insertion: tool 1, needed by the first and last groups but not the middle one, is removed at a group boundary and reloaded (Figure 6).

Lemma 2 (Transition cap)**.** *In any ordering of configurations contained in U, every transition inserts at most $\min(b, q)$ tools. Hence $H_{\text{walk}} \leq (K^* - 1)\min(b, q)$.*

Proof. A transition into C_{k+1} inserts $|C_{k+1} \setminus C_k|$ tools. This is at most $|C_{k+1}| = b$, and also at most $|U \setminus C_k| = q$, since $C_{k+1} \subseteq U$ and $|C_k| = b$. A covering configuration inside U exists for every group, so some minimum-cardinality grouping uses only such configurations, and H_{walk} is at most its cost. □

These two lemmas give the section's central guarantee. Lemma 2 caps the heuristic's absolute cost, Corollary 1 floors the optimum, and dividing one by the other bounds how far from optimal the method can stray.

Theorem 2 (Worst-case guarantee)**.** *For every instance with $K^* \geq 2$,*

$$\frac{H}{Z^*} \;\leq\; \frac{H_{\text{walk}}}{Z^*} \;\leq\; \frac{(K^*-1)\min(b,q)}{\max(K^*-1,\;q)} \;\leq\; \min\Big(b,\;K^*-1,\;q\Big).$$

Proof. The first inequality is Proposition 2. For the second, divide Lemma 2 by $Z^* \geq \max(K^*-1, q) \geq 1$ (Corollary 1). For the third, $(K^*-1)q/\max(K^*-1,q) = \min(K^*-1,q)$ together with $\min(b,q) \leq q$ gives the $\min(K^*-1,q)$ term, while $(K^*-1)/\max(K^*-1,q) \leq 1$ together with $\min(b,q) \leq b$ gives the b term. □

This subsumes the trivial b-approximation that follows from $H_{\text{walk}} \leq b(K^*-1)$ and $Z^* \geq K^*-1$, which was the bound the internship began with. It tightens automatically on instances with few groups, with a small tool excess, or with a small magazine.

Corollary 3 (Zero-gap classes, every b)**.** (i) *If $K^* = 2$ the gap is zero, and indeed $H = H_{\text{walk}} = Z^* = q$.* (ii) *If $|U| \leq b+1$ the gap is zero.* (iii) *If $K^* = 3$ then $H/Z^* \leq 2$.*

Proof. All three are instances of Theorem 2, whose ratio bound is $K^*-1 = 1$, $q \leq 1$ and $K^*-1 = 2$ respectively. For the equality in (i), pad the first group's configuration to size b using tools from the second group's requirement, and pad the second from the first. The single transition then inserts exactly the q tools of U missing from the first configuration, so $H_{\text{walk}} \leq q \leq Z^* \leq H \leq H_{\text{walk}}$ by the coverage bound. □

A positive gap therefore requires $K^* \geq 3$.

Corollary 4 (Small optima are gap-free)**.** *If $Z^* \leq 2$ then the gap is zero, for every b and every K^*.*

Proof. $Z^* \geq q$, so $q \leq 2$. If $q \leq 1$ the gap vanishes by Corollary 3(ii). If $q = 2$ then $Z^* = q$, and the walk argument of Appendix A yields a walk of at most $q+1 = 3$ configurations, each serving a job, exhibiting a feasible grouping of that cardinality; for $K^* \leq 2$ the gap vanishes by Corollary 3(i), and for $K^* = 3$ the walk grouping is itself of minimum cardinality and has cost Z^*, forcing $H_{\text{walk}} = Z^*$. □

Corollary 5 ($Z^* \leq 3$ forces a walk gap of at most one)**.** *If $Z^* \leq 3$ then $H_{\text{walk}} - Z^* \leq 1$, for every b and every K^*.*

Proof. Write $Z^* = q + R_{\text{opt}}$, so $q \le 3$. If $q \le 1$ or $K^* \le 2$ the gap is zero by Corollary 3. Assume $q \in \{2, 3\}$ and $K^* \ge 3$. The excised optimal walk has $p \le Z^* + 1 \le 4$ configurations, each serving a job, and exhibits a feasible p-grouping, so $K^* \le p \le 4$. If $K^* = 4$ then $p = K^*$, the walk's grouping is of minimum cardinality and has cost Z^*, forcing $H_{\text{walk}} = Z^*$. There remains $K^* = 3$. For $(q, R_{\text{opt}}) = (3, 0)$ the merge argument in Appendix A gives a gap of at most one. For $(q, R_{\text{opt}}) = (2, 1)$ the bound of Proposition 7 reads $\max\Big(0, \min(2, \lfloor (2b-2)/3 \rfloor) - 1\Big) \le 1$. □

4.3 The extremal behaviour at three groups

Corollary 3 places every positive-gap instance at $K^* \ge 3$. This subsection works out the case $K^* = 3$, which is where the extremes occur. All results here are about the walk value; Section 4.4 then asks what the KTNS step does to them.

Proposition 6 (Exact walk cost at $K^* = 3$)**.** *If $K^* = 3$ then, with configurations restricted to subsets of U,*

$$H_{\text{walk}} \;=\; q \;+\; \min\;\min\Big(x_{ab},\, x_{bc},\, x_{ca}\Big), \qquad x_{uv} = \Big|(C_u \cap C_v) \setminus C_w\Big|,$$

where the outer minimum runs over feasible 3-groupings and choices of their configurations, and w denotes the third index in each case.

Proof. The three configurations of a feasible 3-grouping are distinct, since two equal ones would merge their groups and contradict $K^* = 3$, and together they cover U. For a path (C_a, C_b, C_c), splitting the cost into first-time insertions and re-insertions (Lemma 1) gives cost $q + |(C_a \cap C_c) \setminus C_b|$: the re-inserted tools are exactly those of C_c present in C_a but absent from C_b. Minimising over the three possible middle configurations selects the smallest of the three overlap terms, and H_{walk} then minimises over groupings and configuration choices. □

This turns grouping selection at $K^* = 3$ into an explicit criterion: choose the grouping whose configurations have the smallest pairwise overlap outside the third. Section 4.7 returns to it.

Lemma 3 (Overlap counting)**.** *Any three configurations $C_a, C_b, C_c \subseteq U$ of size b with $C_a \cup C_b \cup C_c = U$ satisfy* $\min(x_{ab}, x_{bc}, x_{ca}) \le \lfloor (2b - q)/3 \rfloor$.

Proof. Count tools by how many of the three configurations contain them. Let a be the number in exactly one, $\sum x := x_{ab} + x_{bc} + x_{ca}$ the number in exactly two, and c_3 the number in all three. Counting tools gives $a + \sum x + c_3 = b + q$, and counting memberships gives $a + 2\sum x + 3c_3 = 3b$. Subtracting, $\sum x = 2b - q - 2c_3 \le 2b - q$. The minimum of three numbers is at most their mean, and it is an integer. □

Proposition 7 (The walk gap at $K^* = 3$)**.** *Let $K^* = 3$ and $R_{\text{opt}} = Z^* - q$. Then*

$$H_{\text{walk}} - Z^* \;\le\; \max\Big(0,\; \min\Big(q,\; \lfloor (2b-q)/3 \rfloor\Big) - R_{\text{opt}}\Big).$$

Consequently $H_{\text{walk}} - Z^* \le 1$ holds (i) *for every instance with $b \le 4$;* (ii) *whenever $2b \le q + 3R_{\text{opt}} + 5$;* (iii) *whenever $R_{\text{opt}} \ge q - 1$; and* (iv) *at $Z^* = q = 3$ when two of the four walk groups have combined requirement fitting one magazine. The bound can therefore fail only in the corner $2b \ge q + 3R_{\text{opt}} + 6$ with $R_{\text{opt}} \le q - 2$, which forces $b \ge 5$.*

Proof. The configurations of any feasible 3-grouping cover U, so Proposition 6 with Lemma 3 gives $H_{\text{walk}} \le q + \lfloor (2b-q)/3 \rfloor$, while Lemma 2 gives $H_{\text{walk}} \le 2\min(b, q) \le 2q$. Subtracting $Z^* = q + R_{\text{opt}}$ yields the bound. For (i) at $b = 3$, $\lfloor (6-q)/3 \rfloor \le 1$ for every $q \ge 1$. At $b = 4$ the bound is at most 1 for $q \ge 3$; for $q = 2$, either $R_{\text{opt}} \ge 1$ and the bound is at most 1, or $Z^* = q \le 2$ and the gap vanishes by Corollary 4; and $q \le 1$ is Corollary 3(ii). Parts (ii) and (iii) are read off the bound, and part (iv) is proved in Appendix A. □

Proposition 8 (A bound for every K^*)**.** *For every instance with $K^* \ge 2$,*

$$H_{\text{walk}} - Z^* \;\le\; \max\Big(0,\; \Big\lfloor \frac{q\big(b(K^*-1) - q\big)}{b+q} \Big\rfloor - R_{\text{opt}}\Big).$$

Proof. Fix a minimum-cardinality grouping. Its configurations $C_1, \ldots, C_{K^*} \subseteq U$ are pairwise distinct and cover U. In an ordering σ the path cost is $q + \sum_t (\text{runs}_\sigma(t) - 1)$, where $\text{runs}_\sigma(t)$ counts the maximal blocks of consecutive configurations containing t, since every block after the first begins with a re-insertion (Lemma 1). Let d_t be the number of configurations containing t, so that $\sum_t d_t = K^* b$ over $b + q$ tools. Under a uniformly random ordering, $\mathbb{E}[\text{runs}(t) - 1] = (d_t - 1)(K^* - d_t)/K^*$, which is concave in d_t, so by Jensen's inequality the expected total excess is at most $q\big(b(K^*-1) - q\big)/(b+q)$. Some ordering achieves at most the mean, the total is an integer, and subtracting $Z^* = q + R_{\text{opt}}$ gives the claim. □

At $K^* = 3$ this and Proposition 7 are incomparable and both are attained at the 6-ring. Proposition 8 grows linearly in K^* and is unconditional.

The next result shows that the natural guess gap $\le K^* - 2$, which held on all the evidence collected before this internship, is false for the walk value. All the earlier evidence lay at $b \le 4$, which is exactly the region Proposition 7 proves.

Proposition 9 (The $K^* - 2$ bound fails for the walk value)**.** *There are instances with $K^* = 3$ and $H_{\text{walk}} - Z^* = 2$. One, with $b = 5$ and $U = \{1, \ldots, 9\}$, is*

$$W: \quad T = \Big(\{1,2,3,5,8\},\; \{1,5,6\},\; \{2,6\},\; \{3,7,9\},\; \{4,5,6,9\}\Big),$$

with $q = 4$, $Z^* = 4$ and $H_{\text{walk}} = 6$, both values obtained by exhaustive computation. The instance lies exactly on the boundary of the regimes of Proposition 7, at $2b = q + 3R_{\text{opt}} + 6$, and attains that proposition's bound with equality.

Observation 1. On the same instance W, the KTNS step gives $H = 5$, so the heuristic gap is one and the guess gap $\leq K^* - 2$ is not contradicted by it.

Conjecture 1 (The 4/3 ratio at $b = 3$)**.** For every instance with $b = 3$, $H/Z^* \leq 4/3$.

The evidence is a complete enumeration of two families. The first is every instance in which each job requires exactly two tools, with $b = 3$, at most six tools and at most six jobs: 10,691 instances, of which the maximum ratio is exactly 4/3, attained at the 6-ring in the walk model. The second is every instance with $b = 3$ whose jobs require two or three tools, with at most five tools and at most five jobs: a further 6,065 instances, none exceeding 4/3.

Proposition 10 (The 4/3 bound holds whenever $K^* \leq 3$)**.** *Every instance with $b = 3$ and $K^* \leq 3$ satisfies $H_{\text{walk}}/Z^* \leq 4/3$, and hence $H/Z^* \leq 4/3$. Conjecture 1 is therefore open only for $K^* \geq 4$.*

Proof. For $K^* \leq 2$ the gap is zero (Corollary 3). For $K^* = 3$, Proposition 7(i) gives a walk gap of at most one, and a positive gap forces $Z^* \geq 3$ (Corollary 4), so $H_{\text{walk}}/Z^* = 1 + (H_{\text{walk}} - Z^*)/Z^* \leq 1 + 1/3$. Proposition 2 carries the bound to H. □

The reduction to $K^* \geq 4$ is not vacuous. Positive-walk-gap instances with $b = 3$ and $K^* = 4$ exist; one is $T = (\{1, 3, 6\}, \{1, 4, 7\}, \{2, 3\}, \{2, 5\}, \{2, 7\})$, with $Z^* = 4$ and $H_{\text{walk}} = 5$. No ratio above 4/3 has been found in that regime.

4.4 What the KTNS step does to the walk gap

Everything in Section 4.3 bounds H_{walk}. By Proposition 2 those bounds also hold for H. The question this subsection asks is how much slack that inequality hides. The answer is: a great deal, and on exactly the instances the literature uses as examples.

Observation 2 (The ring family has no heuristic gap)**.** For the k-ring at $b = 3$ with $k = 3, \ldots, 9$, the heuristic is optimal: $H = Z^*$ in every case. The walk value is one above the optimum for $k = 6, \ldots, 9$. Table 2 gives the values.

This is why the distinction of Section 2.5 cannot be dropped. The 6-ring is the standard example of a positive gap for this heuristic, and for the heuristic as defined it has no gap at all. The same holds for the $K^* = 4$ instance at the end of Section 4.3, whose walk gap is one and whose heuristic gap is zero, and for the witness W of Proposition 9, whose walk gap is two and whose heuristic gap is one.

Positive heuristic gaps do exist. They are simply rarer and smaller than the walk model suggests.

k	K^*	Z^*	H_{walk}	walk gap	H	heuristic gap
3	1	0	0	0	0	0
4	2	1	1	0	1	0
5	3	2	2	0	2	0
6	3	3	4	1	3	0
7	4	4	5	1	4	0
8	4	5	6	1	5	0
9	5	6	7	1	6	0

Table 2: The k-ring at $b = 3$, free-initial costs, computed exhaustively. The walk value separates from the optimum at $k = 6$ and stays one above it. The value the heuristic actually returns equals the optimum for every k tested. The mechanism is the one drawn in Figure 6.

Observation 3 (The smallest sub-optimal instance)**.** The instance with $b = 4$, four jobs and $U = \{1, \ldots, 7\}$ given by

$$T = \Big(\{1, 2\},\ \{3, 4\},\ \{1, 3, 5, 6\},\ \{2, 5, 6, 7\}\Big)$$

has $K^* = 3$, $Z^* = 3$ and $H = 4$, so the heuristic is suboptimal on it. An exhaustive search over all instances with at most four jobs finds no smaller one.

Observation 4 (Positive heuristic gaps are rare and small)**.** Draw instances with a uniformly random requirement of admissible size for each job. In 1,260 instances with three to five jobs, four to eight tools and capacity between two and five, drawn as three independent samples of 420, *none* has $H > Z^*$. Extending the census to larger instances finds them, but only just: 3 of 1,260 at six jobs and 2 of 1,260 at seven. Every positive heuristic gap found anywhere in this study equals one.

Capacity, not job count, is what admits them. A targeted census at fixed capacity finds no positive gap in 11,000 instances at $b = 2$, spread over four to eight jobs, while at $b = 3$ they occur at a rate of roughly one in a thousand and are present at every job count from five upwards. The smallest found at $b = 3$ has five jobs and seven tools,

$$T_1 = \{4, 6\}, \quad T_2 = \{0, 1, 3\}, \quad T_3 = \{1, 6\}, \quad T_4 = \{0, 5, 6\}, \quad T_5 = \{4, 5, 7\},$$

with $K^* = 4$, $Z^* = 4$ and $H = 5$.

Remark 3 (A correction)*.* An earlier draft of this report stated that every instance carrying a positive heuristic gap has $b \geq 4$, on the strength of the 1,260-instance census alone. The targeted search above refutes that: the witness just given has $b = 3$. The census had missed it because positive gaps at $b = 3$ are about a thousand times rarer than the sample could resolve. What survives is the weaker and now better-supported statement that $b = 2$ admits none in eleven thousand draws, and that the gap needs a magazine

large enough to hold more than one job's requirements with room to spare.

The capacity condition matters. A positive heuristic gap needs the magazine to be large relative to the jobs. That is the regime in which Burger et al. [7] report the decomposition degrading: as the difference between the magazine size and the largest job size grows, the quality of the heuristic's solutions decreases. Their finding is empirical and industrial, this one is combinatorial and exhaustive, and they agree.

Unboundedness survives, but not with the family the walk model suggests.

Proposition 11 (Unbounded walk gap)**.** *Let R_g consist of g tool-disjoint copies of the 6-ring, so that $b = 3$ and $|U| = 6g$. Then $Z^* = 6g - 3$ and $H_{\text{walk}} = 7g - 3$, so the walk gap g grows without bound, while the walk ratio $(7g - 3)/(6g - 3)$ decreases from $4/3$ towards $7/6$.*

The proof is in Appendix A. It pairs the coverage bound with an explicit schedule attaining it, and counts the heuristic's cost as g within-copy contributions of four and $g - 1$ inter-copy transitions of three.

By Observation 2 this construction says nothing about H: the ring has no heuristic gap, so copies of it have none either. Replacing the seed repairs the construction.

Observation 5 (Unbounded heuristic gap, with a different seed)**.** Let S be the four-job instance of Observation 3, which has $b = 4$, $|U| = 7$ and heuristic gap one, and let S_g consist of g tool-disjoint copies of it. Exhaustive computation gives

g	$\lvert U\rvert$	K^*	Z^*	H	$H - Z^*$
1	7	3	3	4	1
2	14	6	10	12	2

so $Z^* = 7g - 4 = |U| - b$ and the heuristic gap equals g on both.

Conjecture 2. For every $g \geq 1$ the instance S_g of Observation 5 has $Z^* = 7g - 4$ and $H - Z^* = g$. In particular the heuristic gap is unbounded.

Supporting argument. The optimum is immediate: the coverage bound gives $Z^* \geq 7g - 4$, and processing the copies one after another, each by its own optimal schedule, attains it, because consecutive copies are tool-disjoint and so the magazine must be rebuilt in full between them. For H the difficulty is that a group may in principle contain jobs from two different copies, since two jobs requiring two tools each can together require exactly $b = 4$. Verifying that no such grouping helps is what the computation above does, and what a proof would have to establish in general. □

Finally, the phenomenon behind all of this is best stated as a question, since it is the natural one to ask next and it has no answer here.

Problem 1. Characterise the instances on which the KTNS step closes the walk gap, that is, on which $H < H_{\text{walk}}$. Equivalently: when does re-optimising the loading over the whole flattened order recover an optimum that no single magazine per group can reach?

The evidence assembled here is that this happens very often, and that the walk model therefore substantially overstates the weakness of the standard method.

4.5 What the gap does not depend on

The results above are quantitative. This subsection asks from which combinatorial structure of an instance the gap could, even in principle, be read. The natural object is the family of feasible groups.

Definition 8 (Group complex and circuit clutter)**.** Let $\mathcal{F} = \{S \subseteq J : |\bigcup_{j \in S} T_j| \leq b\}$. Since $|T_j| \leq b$ and membership is preserved under taking subsets, $\mathcal{F}$ is an independence system on J. Its inclusion-minimal non-members, the *circuits*, form a clutter $\mathcal{C}$.

Three plain ideas sit in that definition. That $\mathcal{F}$ is an independence system means only that feasibility is preserved downwards: remove a job from a group that fits, and it still fits. The circuits are the minimal ways to fail: a set of jobs whose tools overflow the magazine although every proper subset fits. A clutter is a family no member of which contains another, which the circuits form automatically.

Proposition 12 (K^* is a hypergraph chromatic number)**.** *A set of jobs is a feasible group if and only if it contains no circuit. Hence K^* is the chromatic number of the hypergraph $(J, \mathcal{C})$. The* conflict graph*, whose edges join pairs of jobs that cannot share a configuration, consists exactly of the two-element circuits, so its chromatic number is at most K^* and may be strictly smaller. For $T_1 = \{1, 2\}$, $T_2 = \{1, 3\}$, $T_3 = \{1, 4\}$ with $b = 3$ there is no conflicting pair at all, yet the three jobs together require four tools, so they form a circuit and $K^* = 2$.*

Proof. An infeasible set contains a minimal infeasible subset and feasibility is preserved downwards, which gives the first claim. Colour classes of a proper colouring of $(J, \mathcal{C})$ are then exactly feasible groups. Two-element circuits are precisely the conflicting pairs, and every proper colouring of the hypergraph properly colours the graph. The stated instance computes as claimed. □

Figure 8 draws the example. The point it makes is that pairwise conflicts undercount: three jobs can be pairwise compatible and jointly infeasible, so the true obstruction is the hypergraph of all minimal clashes.

Proposition 13 (No matroid structure)**.** *$\mathcal{F}$ is not a matroid in general. For $T_1 = \{1\}$, $T_2 = \{2\}$, $T_3 = \{3, 4\}$ with $b = 2$, both $\{3\}$ and $\{1, 2\}$ are independent, but neither $\{1, 3\}$*

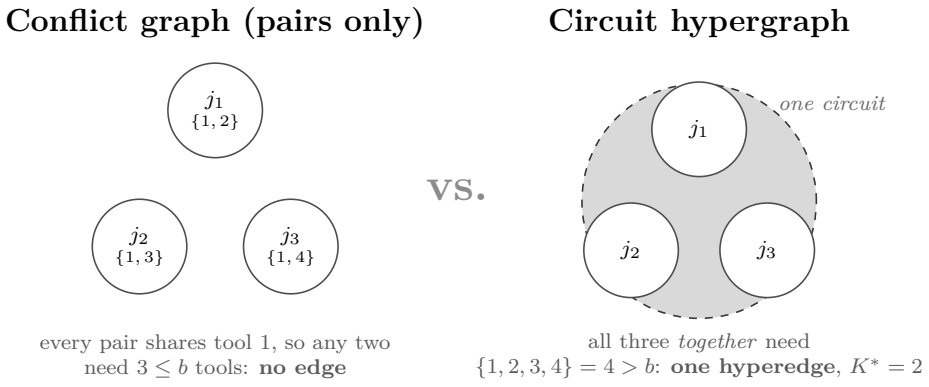

Figure 8: Why K^* is read off the hypergraph of all minimal clashes and never off the graph of pairwise ones. On this instance no two jobs conflict, because any two of them need only three tools, which fit. All three together need four and overflow the magazine, so they form a circuit, and that single circuit already forces two groups.

nor $\{2,3\}$ *is, so the exchange axiom fails. Greedy and matroid-union arguments are therefore unavailable without further hypotheses.*

Proof. The relevant unions have sizes 2, 2, 3 and 3. □

Proposition 14 (The circuit clutter does not determine the gap)**.** *Two instances with* $b = 4$, $|U| = 7$, *four jobs, isomorphic circuit clutters and equal* $K^* = 3$ *and* $Z^* = 3$ *can have different gaps:*

$$I_0 : T = \big(\{1,3,7\},\ \{2,3,4\},\ \{3,4,5,6\},\ \{4,5,6\}\big),$$
$$I_1 : T = \big(\{1,2,4,7\},\ \{2,5,6,7\},\ \{3,5\},\ \{4,6\}\big).$$

In both, every pair of jobs conflicts except the last two, and no larger set is a minimal clash. Yet I_0 *has gap zero and* I_1 *has gap one, in both the walk value and the heuristic value.*

Proof. The pairwise union sizes are checked directly. Exhaustive enumeration over sequences and over minimum-cardinality groupings gives $Z^* = 3$ and $H = H_{\text{walk}} = 3$ for I_0, and $Z^* = 3$ and $H = H_{\text{walk}} = 4$ for I_1. □

No criterion that reads only the circuit clutter, and a fortiori none that reads only the conflict graph, can therefore decide when the gap vanishes. Tool-level structure is essential, and that is exactly the level at which Proposition 6 operates: its overlap sizes x_{uv} are precisely the data the clutter forgets.

On the covering side, however, the clutter is the right object, and blocking duality makes this precise.

Proposition 15 (Circuit–group blocking identity)**.** *On ground set* J*, the blocker of the circuit clutter, meaning the clutter of its minimal transversals, is the family of minimal complements of maximal feasible groups.*

Proof. A set meets every circuit if and only if its complement contains no circuit, if and only if its complement is a feasible group. Minimal transversals therefore correspond to maximal feasible complements. □

This has a concrete consequence for the next section. Grouping branch-and-price rests on a set-covering relaxation, and it is fast only when that relaxation is *ideal*, meaning its fractional optimum is already integral. The next result exhibits the smallest family where it is not.

Proposition 16 (Odd rings are not ideal)**.** *Consider the clutter of job clusters over the maximal feasible groups; its blocker is the clutter of minimal group-covers, so K^* is the blocker's minimum size. On the k-ring with $b = 3$ and $k \geq 4$, the fractional cover equals $k/2$ while $K^* = \lceil k/2 \rceil$. The covering relaxation is therefore tight for every even k and has integrality gap $\frac{1}{2}$ for every odd k.*

Proof. For $k \geq 4$ the maximal feasible groups are exactly the pairs of adjacent jobs. Weight $\frac{1}{2}$ on each covers every job exactly once, so the fractional cover is at most $k/2$. The dual packing $y_j = \frac{1}{2}$ is feasible, since no group holds more than two jobs, so by linear-programming duality the fractional cover equals $k/2$. The integer optimum is $\lceil k/2 \rceil$. □

The hypothesis $k \geq 4$ is needed: at $k = 3$ all three jobs require only the three tools $\{1, 2, 3\}$, so the whole job set is one feasible group and $K^* = 1$.

This locates a first provable obstruction for the set-covering relaxation behind grouping branch-and-price [13, 38]: odd circular structure, in exact parallel to odd holes in perfect-graph theory [10].

It is natural to ask whether it is the *only* obstruction. At $b = 3$ with every job requiring exactly two tools the question is finite, because the family is then a family of graphs: a tool is a vertex, a job is an edge, and a feasible group is a set of edges spanning at most three vertices. The k-ring is the cycle C_k. Every such instance up to isomorphism on at most six tools can therefore be enumerated and tested, and the answer is negative.

Proposition 17 (A non-ring obstruction)**.** *At $b = 3$ the three-job star*

$$T_1 = \{a, b\}, \qquad T_2 = \{a, c\}, \qquad T_3 = \{a, d\}$$

has fractional cover $\frac{3}{2}$ and $K^* = 2$. It contains no odd cycle, so odd-ring structure among the tools is not necessary for the covering relaxation to be non-integral.

Proof. Any two of the three jobs together require three tools and form a feasible group; all three require four and do not. The maximal feasible groups are therefore the three pairs, and each job lies in exactly two of them, so weight $\frac{1}{2}$ on each pair covers every job

and the fractional cover is at most $\frac{3}{2}$. The dual packing $y_j = \frac{1}{2}$ is feasible because no group holds more than two jobs, so by linear-programming duality the fractional cover equals $\frac{3}{2}$. No single group holds all three jobs, so $K^* = 2$. The tool graph is the star $K_{1,3}$, which is bipartite. □

Observation 6 (How common the obstruction is)**.** Of the 151 instances of this family with at most six tools, exactly 76 have a non-integral covering relaxation. Eleven of those are bipartite. The star of Proposition 17, with three jobs, is the smallest non-integral instance of the family; the smallest ring witness has five.

What the star and the odd ring share is not circular structure among the *tools* — the star has none — but odd structure among the *jobs*: in both cases three jobs are pairwise compatible and not jointly compatible. That is the odd hole, seen in the compatibility hypergraph on jobs rather than in the graph on tools, and it is where a characterisation should be sought. Section 8.2 records what remains open.

4.6 The setup-cost spectrum

In practice a machine stop costs time beyond the individual tool exchanges. This subsection shows that the SSP and the JGP are the two ends of a single parameterised problem, with the transition governed by the gap itself. Burger et al. [7] pose exactly this question at the end of their paper.

Charge, in addition to the tool switches, a cost $\rho \geq 0$ for every *configuration change*. The objective becomes

$$\text{switches} \;+\; \rho \cdot (\text{number of configuration changes}),$$

where ρ is the ratio of the fixed stop time to the time of one tool exchange. A grouping into p configurations makes $p - 1$ changes, so the setup term alone is minimised by the Job Grouping Problem. As ρ grows it dominates and pulls the optimum onto minimum-cardinality groupings.

Theorem 3 (Setup-cost collapse)**.** *If $\rho > H_{\text{walk}} - Z^*$, then every optimal solution of the setup-augmented objective uses a minimum-cardinality grouping, and the two-phase heuristic is exact for that objective. The collapse threshold therefore satisfies $\rho_c \leq H_{\text{walk}} - Z^*$, and for the ring family $\rho_c = 1$.*

Proof. Every feasible grouping $\mathcal{P}$ has $|\mathcal{P}| \geq K^*$ and, by Theorem 1, $\gamma(\mathcal{P}) \geq Z^*$, so its augmented cost is at least $\rho(|\mathcal{P}| - 1) + Z^*$. The best minimum-cardinality grouping has augmented cost $\rho(K^* - 1) + H_{\text{walk}}$. If $|\mathcal{P}| > K^*$, the former exceeds the latter whenever $\rho(|\mathcal{P}| - K^*) > H_{\text{walk}} - Z^*$, which holds for every such $\mathcal{P}$ as soon as $\rho > H_{\text{walk}} - Z^*$. No

grouping of larger cardinality is therefore optimal, and the heuristic, which selects the cheapest ordering of a minimum-cardinality grouping, attains the optimum. □

The threshold must be read in the walk value. Using the heuristic value instead makes the statement false, because on the 6-ring $H - Z^* = 0$ while a four-configuration grouping still wins for small positive ρ.

Lemma 4 (The other end of the spectrum)**.** *If $\rho < 1/(n-1)$, every optimum of the setup-augmented objective is a switch-optimal SSP schedule.*

Proof. A schedule makes at most $n-1$ configuration changes. Let S^* be switch-optimal. Any schedule with at least Z^*+1 switches has augmented cost at least $Z^*+1 > Z^*+\rho(n-1)$, which is at least the augmented cost of S^*. No such schedule is therefore optimal. □

Together these pin both ends. For $\rho < 1/(n-1)$ the augmented problem is the SSP. For $\rho > H_{\text{walk}} - Z^*$ its optima use minimum-cardinality groupings and the two-phase heuristic is exact, so the problem has collapsed onto the JGP. The transition is confined to the interval $[\,1/(n-1),\ H_{\text{walk}} - Z^*\,]$, which is to say it is governed by the gap itself.

Remark 4 (On practical values of ρ)*.* It is natural to ask where real manufacturing sits on this spectrum. Answering that requires measured values of the stop time and the per-tool exchange time in a comparable setting, and no source giving both could be found during this internship. The spectrum result is therefore stated as it is proved, as a statement about the parameter ρ, and no claim is made about where practice falls on it. Obtaining such measurements would settle a question that Theorem 3 otherwise leaves open.

4.7 Which grouping to choose

Theorem 1 turns the heuristic's weakness into a concrete algorithmic question. The optimum is attained by *some* feasible grouping, and minimum-cardinality ones may miss it (Example 5). On which grouping, then, should the sequencing phase be run?

Admitting a single extra group already recovers the ring optimum in the walk model. In general the trade-off is between more configuration changes and cheaper individual transitions. At $K^* = 3$, Proposition 6 makes the criterion explicit: choose the grouping whose configurations have the smallest pairwise overlap outside the third. For general K^*, characterising the groupings that attain $\min_{\mathcal{P}} \gamma(\mathcal{P})$, or giving an efficient rule that improves on the minimum-cardinality choice, is open.

Two things follow from Section 4.4. The first is that the practical value of such a rule is smaller than the walk model suggests, since the KTNS step already recovers the optimum on most instances where the walk model loses it. The second is that the instances where a rule would help are identifiable in advance by the same criterion that identifies positive gaps: a magazine large relative to the jobs.

Method	Source	Formulation	Linear relaxation
F4	Catanzaro et al. [9]	compact, arc-based	in general below q
LSS	Laporte et al. [28]	compact, tool-state	in general below q
SSPMF	da Silva et al. [16]	multicommodity flow	$= q$ when every tool is used
BBC	this work	branch-and-Benders-cut	no relaxation of the loading
PCF, PCF′	this work	position–configuration B&P	$= q$ with the counting rows
PTF	this work	position–transition B&P	$\geq q$, and can exceed it

Table 3: The three compact baselines and the methods of this report, ordered by the strength of their linear relaxation relative to the coverage bound q. Every method is exact; they differ in whether, and by what means, they get past that bound. The entries for F4 and LSS are hedged because the source states only that earlier models fall below q in the great majority of cases [16], not in all.

5 Exact methods

Section 4 showed that the coverage bound $q = |U| - b$ equals the optimum on a large part of the instance space. Section 3 showed that the strongest compact formulation in the literature has a linear relaxation equal to that same quantity. This section asks whether an exact method can do better, and builds two that are designed to.

Why these two methods. A compact integer formulation is the natural first recourse, and three from the literature serve here as baselines. Their relaxations do not exceed the coverage bound, so a branch-and-cut built on any of them is fast where that bound is tight and adrift where it is not. Escaping this requires a method that does not rest on a compact relaxation, and two classical routes qualify, for opposite reasons.

Benders decomposition is the textbook fit when the subproblem is polynomially solvable [43]. The loading problem is such a subproblem, so it becomes an oracle returning exact cuts, and the master's bound is assembled from true subproblem values rather than from a weak relaxation. The algorithm never forms the weak relaxation at all. This places the solver in the lineage of logic-based [25] and combinatorial [11] Benders decomposition.

Branch-and-price attacks the same weakness from the other side. Reformulating over configurations, its position–transition variant carries a relaxation that can provably exceed the coverage bound (Proposition 22), which is the one construction in this report that does. How often it does so on real instances is a separate question, and Section 6.6 answers it.

5.1 Compact formulations and the limit they reach

Three compact formulations from the literature serve as baselines: Formulation 4 of Catanzaro et al. [9], the tool-state model of Laporte et al. [28], and the multicommodity-flow model of da Silva et al. [16]. Section 3.2 describes all three and records the omission of the stronger Formulation 5 as a limitation. All three were reimplemented for this study,

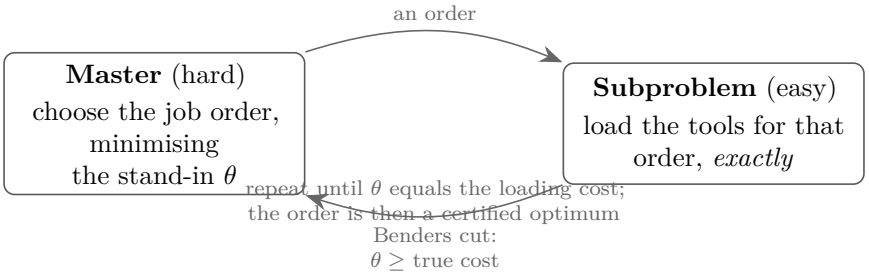

Figure 9: The branch-and-Benders-cut loop. The master picks an order using an optimistic stand-in θ for the loading cost. The subproblem returns the exact loading cost of that order. If θ undershot, one cut lifts it, and that cut is valid for every order, not only the current one. Because the subproblem is polynomial every cut is exact, so the loop halts at a proven optimum. The cuts are separated inside a branch-and-bound tree over the order variables, which is what makes the method a *branch*-and-Benders-cut.

and Section 6.2 describes how they were checked. The last is singled out here by its relaxation.

Proposition 18 (16)**.** *The linear relaxation of the multicommodity-flow formulation equals $|T| - b$, the number of tools minus the magazine capacity. Under the standing assumption that every tool is required by some job, so that $U = T$, this is the coverage bound q of Proposition 4.*

The hypothesis matters and is stated explicitly by the authors. On an instance with unused tools the two quantities differ, and it is q, not $|T| - b$, that bounds the optimum. The instances of the computational study all satisfy the hypothesis.

This result matters twice. It says that the strongest compact relaxation available is no stronger than an elementary counting bound. And since q equals Z^* on a large part of the instance space (Section 4), it predicts that such a model will be fast exactly where the bound is tight and weak where it is not. Section 6.5 tests that prediction directly.

5.2 A branch-and-Benders-cut algorithm

The idea. A schedule is an order together with a loading. Once the order is fixed the minimum loading cost is computed exactly and in polynomial time. The order therefore goes into an integer master problem, and the loading cost is represented by a single continuous variable θ, refined by cuts generated from the exactly solved loading subproblem. In contrast to column generation, whose pricing subproblem for this problem is NP-hard (Section 5.4), the Benders subproblem here is polynomial, so every cut is exact.

Figure 9 shows the loop.

The master problem. Introduce a depot node d with an empty tool requirement, and arc variables $x_{ij} \in \{0,1\}$ for $i, j \in J \cup \{d\}$ with $i \neq j$, where $x_{ij} = 1$ means job j runs immediately after i. The depot marks the start and the end, so the x variables describe a Hamiltonian path through the jobs. The assignment constraints

$$\sum_{i \neq j} x_{ij} = 1 \quad (\forall j), \qquad \sum_{j \neq i} x_{ij} = 1 \quad (\forall i),$$

together with subtour-elimination constraints $\sum_{i,j \in S} x_{ij} \leq |S| - 1$ for $S \subseteq J$, enforce that. The master problem is then

$$\min \ \theta$$

subject to the assignment constraints, the subtour constraints, the bound rows described next, and the Benders cuts generated during the search.

Three families of rows bound θ from below before any cut is generated.

The pairwise bound. At the boundary between consecutive jobs i and j at least $w_{ij} := \max(0, |T_i \cup T_j| - b)$ tools are switched, since the magazine holds b tools but must serve T_i and then T_j. This is the bound of Tang and Denardo [45]. Because the subproblem charges insertions from an empty magazine, the first job also costs at least $|T_j|$ insertions, and that term is carried on the depot arcs. Summing gives the valid inequality

$$\theta \ \geq \ \sum_{i,j \in J} w_{ij}\, x_{ij} \ + \ \sum_{j \in J} |T_j|\, x_{d,j},$$

in which the two contributions are disjoint by position and so may be added.

The coverage row. Every tool of U must be loaded at least once. In the empty-start cost this gives $\theta \geq |U|$ directly. This row was absent from the first implementation, and its absence was the reason the solver performed poorly in the first campaign; Section 6.2 records the correction.

Triplet bounds. The pairwise argument generalises to ordered triples of jobs, giving $O(n^3)$ valid inequalities of the form $\theta \geq w_{ijk}(x_{ij} + x_{jk} - 1)$ with $w_{ijk} = \max(0, |T_i \cup T_j \cup T_k| - b)$. These are optional and their effect is measured in Section 6.4.

Subtour separation is exact by construction: violated constraints are detected at integer candidate solutions by a connected-component search, so no optimal solution is ever removed. Separating them at fractional points would be an acceleration, not a correctness requirement, and is deliberately absent from the measured configuration.

The subproblem. Fix the order and relabel the jobs $1, \ldots, n$ along it. With $g_{t,i} \in [0,1]$ indicating that tool t occupies the magazine at position i, and $s_{t,i} \geq 0$ counting an insertion there, the loading cost is the optimal value of the linear program written in Section 2.8. Its constraint matrix has the consecutive-ones property and is therefore

totally unimodular, so the relaxation is integral and, by Proposition 1, its optimum equals KTNS of the order.

Writing R_i for the tool set of the job at position i, an optimal dual solution prices each requirement "tool t present at position i" by a multiplier $\bar{\alpha}_{t,i}$ and each capacity row by $\bar{\beta}_i \geq 0$. Linear-programming duality then gives the Benders optimality cut

$$\theta \;\geq\; \sum_{i} \sum_{t \in R_i} \bar{\alpha}_{t,i} \;-\; b \sum_{i} \bar{\beta}_i,$$

where the multipliers of the unit upper bounds on the $g_{t,i}$ contribute a further term, suppressed here. The requirement multipliers carry the dependence on the order, through which job occupies each position, so the right-hand side is linear in the master's variables. Because the dual feasible region does not depend on the order, the cut is valid for every order, which is the defining property of a Benders cut.

One consequence of using the linear program rather than the KTNS rule is worth recording. Crama et al. [14] show that KTNS is invalid when tools have different setup times, while the same linear program remains exact because its matrix remains an interval matrix. The subproblem used here is that linear program. The solver therefore extends unchanged to non-uniform per-tool switching costs, a regime in which the KTNS-based framing of the problem does not apply. This was not tested experimentally and is stated as a property of the construction.

Cut families. Two kinds of cut are separated, and the computational study switches each on and off independently.

Benders optimality cuts are the dual cuts above, separated at integer master solutions and, optionally, at fractional ones. The latter are called *fractional cuts* and are added as user cuts.

Combinatorial cuts use only the oracle value. At an integer order $\bar{\sigma}$ with $\mathrm{KTNS}(\bar{\sigma}) = \bar{z}$, the inequality

$$\theta \;\geq\; \bar{z}\Big(1 - \sum_{(i,j) \in A(\bar{\sigma})} (1 - x_{ij})\Big)$$

is valid: it forces $\theta \geq \bar{z}$ when the order is exactly $\bar{\sigma}$, and is vacuous otherwise. It needs no dual computation, and it is a combinatorial Benders cut in the sense of Codato and Fischetti [11].

5.3 Strengthening the Benders solver

Four strengthenings were implemented, each as an independent switch, and each checked against brute force on small instances before any campaign run so that a strengthening can never silently return a wrong optimum. All four are classical elements of the toolbox surveyed by Rahmaniani et al. [43].

Fractional Benders cuts. The optimality cut is derived at an integer order, but nothing prevents separating the same cut at the fractional solutions produced at a node before branching. The intention is to attack tailing-off: an integer-only scheme learns about the loading cost only once an order is fully committed, whereas a fractional cut extracts the same dual information from a partial order and removes it from the relaxation at once.

Whether that pays is an empirical question, and Section 6.4 answers it.

A strong incumbent at the root. A branch-and-bound search prunes in proportion to the quality of its incumbent. A compact form of the hybrid genetic search of Mecler et al. [32] is run once, at the root: a population of constructive seeds, being nearest-neighbour orders on the pairwise weights w_{ij} and a largest-tool-first order, each improved by a variable-neighbourhood local search using adjacent swaps and short segment relocations, with every candidate scored by the exact loading oracle, evolved under order crossover with a diversity guard.

The resulting order is used twice. As a warm start it gives the solver an incumbent from node zero. And it seeds a Benders cut: the loading dual is solved once for the heuristic order and its cut added before the first relaxation, which is the initial-cut-pool acceleration of Rahmaniani et al. [43].

A conflict-graph bound. The master's root bound is pairwise plus coverage, and both are visible to the linear relaxation. The combinatorial structure they miss lives in the conflict graph G of Section 4.5, whose vertices are the jobs and whose edges join pairs that cannot share a configuration. A clique of G is a set of pairwise incompatible jobs, which must occupy that many distinct configurations, so $K^* \geq \omega(G)$; more generally $K^* \geq \chi(G)$, and where G is non-bipartite one has $\chi(G) \geq 3$ even when $\omega(G) = 2$. The resulting constant row $\theta \geq (\chi_{\mathrm{lb}} - 1) + \min(b, |U|)$ is added whenever it strictly exceeds the coverage row. This is the simplest of the conflict-graph inequalities studied by Atamtürk et al. [3]. Its reach is narrow by construction: on most instances the coverage row already dominates it, and it can bite only where the grouping structure, rather than the tool count, is what forces switches.

Pareto-optimal cut lifting. When the loading dual has several optimal solutions, which a degenerate transportation-type problem routinely does, the cuts they yield are not equally strong. Choosing the deepest is the Pareto-optimal cut of Magnanti and Wong [31]: among the alternative dual optima, select the one maximising the cut's value at a fixed interior core point of the master. Papadakos [39] makes this practical by observing that the core point need not be tied to the current relaxation and may simply be updated as a convex combination of the points visited. That form is used here. A core point is kept

in the relative interior of the master polytope, initialised from the heuristic order blended with the barycentre; at each fractional separation the loading dual is solved a second time at the core point and its cut added alongside the ordinary one, after which the core point is drawn halfway towards the current relaxation point. Adding the core-point cut in addition to, never instead of, the ordinary cut keeps the scheme valid by construction.

5.4 Position-indexed formulations

The configuration walk of Section 2.3 is the most faithful exact picture of the SSP, but as a model it has two flaws. There are far too many configurations to enumerate, and forbidding the subtours that a valid walk must avoid takes exponentially many constraints.

The position-indexed formulations, proposed by Prof. Colares, remove both flaws by fixing the schedule's length in advance. In place of an open-ended walk they lay out a fixed row of *positions* into which configurations are placed, and ask of each position which configuration sits there and what it costs to change from the previous one. Numbering the positions does two useful things at once. It makes the transition cost a purely local quantity between adjacent slots, so the global subtour constraints disappear, and it caps the model at a number of rows polynomial in n and $|T|$. That last property is the point of the construction: the row set is fixed in advance, so column generation only ever adds columns to rows that already exist.

Why not Miller–Tucker–Zemlin. The obvious alternative is to keep the configuration formulation and replace its subtour constraints by Miller–Tucker–Zemlin potentials [17, 33]. That trades one obstacle for another.

Proposition 19. *A potential per configuration is exact but leaves one potential for each of exponentially many configurations. Aggregating the potentials to one per job is polynomial but no longer exact: it can exclude the optimum.*

Proof. The aggregated form fails under job domination. If $T_i \subseteq T_j$ then every configuration serving j also serves i, so a transition between two such configurations places both i and j at each of its ends, and the aggregated potentials of i and j are forced into a two-cycle that no ordering satisfies. A concrete instance is $b = 3$ with $T = (\{1,4\},\{1,3,4\},\{1\},\{1,2,4\})$, for which the optimum is 1 while the aggregated model values it at 2. This was verified by exhaustive enumeration over covering paths. □

Neither variant therefore yields a tractable exact model, which is what motivates fixing the positions instead.

The position–configuration formulation. Let $y^k_C \in \{0,1\}$ indicate that configuration C occupies position k, for $k = 1,\dots,n$, and let $w^k_t \geq 0$ count the insertion of tool t at

position k:

$$\begin{aligned}
&\min \ \sum_{t,k} w_t^k \\
\text{(P)} \quad &\sum_{C} y_C^k = 1, && k = 1, \dots, n; \\
\text{(Cov)} \quad &\sum_{k} \sum_{C \supseteq T_j} y_C^k \geq 1, && j \in J; \\
\text{(W)} \quad &w_t^k \geq \sum_{C \ni t} \left(y_C^k - y_C^{k-1} \right), && t \in T,\ k \geq 2; \\
\text{(T)} \quad &\sum_{k \geq 2} w_t^k + \sum_{C \ni t} y_C^1 \geq 1, && t \in U.
\end{aligned}$$

Constraint (P) places one configuration per position, (Cov) serves every job, (W) charges an insertion whenever a tool enters the magazine, and (T) records that every used tool is either present at the first position or inserted later. The configuration variables are generated as columns; the rows number $O(n|T|)$ and never change. Call this formulation PCF.

Proposition 20. *PCF is exact: its integer feasible points are exactly the SSP schedules, with equal cost. Its linear relaxation* without *the counting rows (T) equals* 0*. With them it equals* q*.*

The proof is in Appendix A.

The vanishing of the plain relaxation is the fractional pathology of Tang and Denardo [45] again: a single convex blend of configurations, repeated at every position, satisfies (P), (Cov) and (W) at zero cost. The counting rows restore the coverage bound, matching the multicommodity model of Proposition 18 with one extra family of rows.

Symmetry reduction. PCF carries a large padding symmetry. A schedule using $m < n$ distinct configurations is encoded by repeating configurations to fill all n positions, and the repeats may sit anywhere, so one schedule has many equal-cost encodings and therefore many distinct nodes in a search tree. Removing this freedom means relaxing (P) to an inequality and forcing the empty positions to a suffix:

$$\text{(P}') \quad \sum_{C} y_C^k \leq 1 \ \ (\forall k); \qquad \text{(G)} \quad \sum_{C} y_C^{k+1} \leq \sum_{C} y_C^k \ \ (k = 1, \dots, n-1),$$

with (Cov), (W) and (T) unchanged. Call this PCF$'$. At an integer point the counts are non-increasing in k, so the used positions form a prefix and the empty ones a suffix, with the cost-free first configuration at position one. The contiguity rows (G) are not decorative: without them an interior empty position would make (W) read the re-entry as a full reload and (T) misattribute the free load.

Proposition 21. *PCF′ is exact, and its linear relaxation equals that of PCF: q with the counting rows, and 0 without. It removes symmetric integer encodings but does not strengthen the bound.*

The proof is in Appendix A. The separation of concerns is deliberate: PCF′ attacks symmetry, and only the next formulation attacks the bound.

The position–transition formulation. To exceed the coverage bound the formulation indexes *transitions* rather than positions. Augment the configurations with an absorbing tool-less node, used to pad schedules that change configuration fewer than $n-1$ times, and let $z^k_{C,C'} \geq 0$ be a column for the step from position k to position $k+1$ that leaves C and enters C'. The formulation is

$$\min \sum_{k=1}^{n-1} \sum_{(C,C')} d(C,C')\, z^k_{C,C'}$$

$$\text{(S)} \quad \sum_{(C,C')} z^k_{C,C'} = 1, \qquad k = 1,\dots,n-1;$$

$$\text{(F)} \quad \sum_{(C,C')\colon t\in C'} z^k_{C,C'} = \sum_{(D,D')\colon t\in D} z^{k+1}_{D,D'}, \qquad t \in T,\ k = 1,\dots,n-2;$$

$$\text{(Cov)} \quad \sum_{k=1}^{n-1} \sum_{(C,C')\colon T_j\subseteq C} z^k_{C,C'} + \sum_{(C,C')\colon T_j\subseteq C'} z^{n-1}_{C,C'} \ \geq\ 1, \quad j \in J.$$

Constraint (S) selects one transition per step. The flow-consistency rows (F) require, tool by tool, that what enters position $k+1$ at the end of step k is what leaves it at the start of step $k+1$, so the selected columns describe a single coherent walk. Constraint (Cov) serves every job at the configuration of some position, either the tail of a step or the head of the last one. As in PCF the rows number $O(n|T|)$ and are fixed, while the transition columns are generated. Call this PTF.

Proposition 22. *PTF is exact, and its linear relaxation is at least q and can be strictly greater.*

The proof is in Appendix A. The second claim is established by a witness, not by a general argument, which is why it says *can*.

The witnesses are the three instances of Section 6.6 on which the recorded root relaxation exceeds q. PTF is therefore the first relaxation in this family to improve on the elementary bound, and it does so with a fixed, polynomial row set.

The proposition says the relaxation *can* exceed q. It says nothing about how often, or by how much, and Section 6.6 finds that on the standard benchmark the answer to both is: rarely, and by one unit. That leaves the question that decides whether the construction is worth pursuing.

Problem 2. Is there a family of instances on which the PTF relaxation exceeds the coverage bound q by an amount that grows without bound? Equivalently: is the excess of PTF over q bounded by a constant?

Column generation and robust branching. Both PCF′ and PTF have $O(n|T|)$ rows but exponentially many columns, so their relaxations are solved by column generation. Branching on a single column y_C^k is ineffective under the position symmetry and forces the pricing problem to carry a growing list of forbidden columns. The branching used here is instead on the *presence* variables $a_t^k = \sum_{C \ni t} y_C^k \in \{0, 1\}$, which already appear in (W). Fixing a_t^k to 0 or 1 forbids or forces tool t at position k, which in the pricing problem merely shifts one dual weight and leaves the oracle's form unchanged. This is a robust branching rule, and it is why the Ryan–Foster rule of Ryan and Foster [44] is not needed here: that rule exists for set-partitioning masters whose pricing problems variable branching would destroy, and this master is not of that kind.

Pricing PCF′. Let $\alpha_k, \gamma_k \le 0$ and $\pi_j, \mu_{t,k}, \lambda_t \ge 0$ be the duals of (P′), (G), (Cov), (W) and (T). Collecting the coefficient of y_C^k, which enters (W) and (T) through a_t^k, gives the reduced cost

$$\bar{c}(C, k) = \beta_k - \sum_{t \in C} \rho_t^k - \sum_{j:\, T_j \subseteq C} \pi_j,$$

with

$$\rho_t^k = -\mu_{t,k}\,[k \ge 2] \;+\; \mu_{t,k+1}\,[k \le n-1] \;+\; \lambda_t\,[k = 1,\; t \in U],$$

with β_k depending only on the position. For each k the most negative reduced cost is obtained by maximising the configuration-dependent part,

$$\max_{|C|=b} \sum_{t \in C} \rho_t^k + \sum_{j:\, T_j \subseteq C} \pi_j,$$

which is to say: choose b tools so as to collect per-tool rewards plus a bonus for each job whose *entire* requirement lands inside the chosen set. This is a Set-Union Knapsack Problem [22], the same class as the grouping-pricing problem of Crama and Oerlemans [13]. The branching dual enters additively in ρ_t^k, so branching does not change the oracle.

Pricing PTF. For PTF a column is an ordered pair (C, C'), and its reduced cost contains the bilinear term $\sum_t [t \in C' \setminus C](1 - \lambda_t)$ together with chaining terms on the tools of C and C'. The pricing problem cannot therefore separate into a single set: it must choose two b-subsets at once, with $C \ne C'$. The products $x'_t(1 - x_t)$ are linearised by the McCormick inequalities and the problem is solved as a mixed-integer program with $2|T|$ binary variables. This heavier pricing is the algorithmic price of PTF's stronger bound, and the contrast between the two formulations — PCF′ pairing a weak bound with a

cheap single-set oracle, PTF a stronger bound with a costly coupled-pair oracle — is the central trade-off of this section.

Accelerations. Four accelerations were implemented as independent switches, mirroring those of Section 5.3. *Heuristic pricing* answers the pricing question first with a fast greedy subset and calls the exact oracle only to certify that no improving column remains, so exactness is untouched while the costly oracle is skipped on all but the final round. *Multiple pricing* returns several negative-reduced-cost columns per round instead of one. *Warm starting* seeds the initial column pool from a greedy schedule rather than the trivial one-configuration-per-job pool. *Dual stabilisation* [30] smooths successive dual vectors to damp the oscillation that slows column generation on degenerate masters. None of the four alters the relaxation value or the optimum.

5.5 A learned cut-selection prototype

At a given node the Benders solver has several admissible cuts available: a combinatorial cut for each candidate order it evaluates, the coverage row, and, with fractional separation enabled, one cut per violated fractional point. Which to add is currently decided by a fixed rule, and whether a learned policy would do better is a natural question. Prof. Colares raised it during the internship.

A prototype was built, following the cut-selection framework of Tang et al. [46]. Cut selection is cast as a Markov decision process in which the state is the current relaxation solution together with a pool of candidate cuts, each summarised by its violation, support size, right-hand side and mean coefficient magnitude; an action selects one candidate; and the reward is the resulting increase in the dual bound. The policy is a softmax that is linear in those features and is trained by the REINFORCE policy-gradient rule with a return baseline.

Because the Benders solver requires a commercial solver that was unavailable in the development environment, the learning components were implemented and trained on a self-contained cut family rather than on SSP instances. The substitute is knapsack cover cuts: for a knapsack instance and a minimal cover C, the inequality $\sum_{j \in C} x_j \leq |C| - 1$ is valid, the separation is elementary, and the linear relaxations can be solved without a commercial solver. The state, the action, the reward and the policy are the same objects as in the Benders case; only the feature extractor and the source of the candidate cuts differ.

What was measured. The agent was trained for 2,000 episodes at each of six knapsack sizes, with learning rate 0.2 and discount 0.95, and evaluated on a held-out set of instances it had not seen. The quantity compared is the mean tightening of the linear bound

achieved over a fixed number of separation rounds, under the learned policy against uniformly random selection from the same candidate pool. Table 4 gives the result.

knapsack size n	learned policy	random selection	improvement
10	1.3855	0.8734	+58.6%
12	1.1639	0.7014	+65.9%
20	0.7482	0.5021	+49.0%
22	0.6692	0.4568	+46.5%
24	0.6383	0.3888	+64.2%
26	0.5661	0.3279	+72.7%

Table 4: Mean bound tightening on held-out knapsack instances, learned cut selection against random cut selection. One training seed per size. The learning machinery works; the problem is not the SSP.

Two things follow, and only two. The learning machinery is correct and does learn: the policy beats random selection at every size tested, by between 47% and 73%. And that is a statement about knapsack cover cuts, on a single training seed per size, with no repetition to separate the effect from training variance. It says nothing about the SSP. The prototype has not been evaluated on SSP instances, and no claim about its effect on this problem is made anywhere in this report.

Section 8.2 explains why evaluating it on the SSP is unlikely to be worthwhile until the bound question of Section 7 is resolved.

6 Computational study

Computation here is how the theory is tested. The quantity the structural results single out, the coverage bound q, is also the root relaxation of every configuration-based method, so a benchmark that records optima and bounds measures the theory's central variable directly. The campaign also settles a question that predates the theory and motivated the Benders solver in the first place: the SSP has a polynomially solvable loading subproblem, which is the textbook precondition for Benders decomposition, and whether a decomposition can exploit it against modern compact models had not been tested.

What the section asks, and in what order. Each subsection states the question it answers before the measurement that answers it. Section 6.1 fixes the protocol and Section 6.2 asks whether the implementations are sound. Section 6.3 asks how far each method gets; Section 6.4 whether the difficulty is finding good solutions or proving them optimal, and whether the standard strengthenings move it; Section 6.5 what separates the instances a method closes from those it does not. Section 6.6 puts the same question to the branch-and-price prototypes, which report their relaxation value directly and so

answer it without reference to solving times. Section 7 then asks why the answer is what it is.

6.1 Experimental design

Instances. The benchmark uses the standard SSP instance families, taken from the collection assembled by Otiai [38]: the sets of Catanzaro et al. [9], of Crama et al. [14], and four of the sets of Laporte et al. [28]. Every instance in every family is run; none is skipped, and no instance is selected on the basis of an outcome. Table 5 gives the sizes.

Family	Instances	Jobs n	Tools $\lvert T\rvert$	Source
Catanzaro	171	8–40	5–60	Catanzaro et al. [9]
Crama	160	10–40	10–60	Crama et al. [14]
Laporte3	340	8	15–25	Laporte et al. [28]
Laporte4	330	9	15–25	Laporte et al. [28]
Laporte5	340	15	15–25	Laporte et al. [28]
Laporte7	80	10–15	10–20	Laporte et al. [28]
Total	1,421			

Table 5: The instance families. An instance is a matrix together with a magazine capacity. The Crama collection publishes the same matrices at four different capacities, so its 40 matrices give 160 instances; the same convention is applied throughout.

Protocol. Every run of the Benders solver and of the compact baselines is given a single uniform time limit of one hour and 16 gigabytes of memory. There is no early stopping and no per-configuration retirement rule. All other solver parameters are left at their defaults, so that the comparison measures the formulations rather than parameter tuning. The branch-and-price prototypes are run separately, under a ten-minute limit, for the reason given in Section 6.6.

Hardware and software. All runs were executed on the LIMOS SLURM cluster, confined to a homogeneous pool of identical nodes (AMD EPYC 7452, 2×32 cores, 512 GB of memory), so that wall-clock times are comparable across runs. The mixed-integer solver is CPLEX 22.1.1 throughout, pinned to four dedicated threads per run. The branch-and-price prototypes run single-threaded under PySCIPOpt. The campaign was distributed over 120 SLURM array shards and produced 16,994 individual runs.

Configurations. Twelve solver configurations were run. Three are the reimplemented compact baselines: F4 of Catanzaro et al. [9], written `CATZ-F4`; LSS of Laporte et al. [28]; and SSPMF of da Silva et al. [16]. The remaining nine are the Benders solver, in an ablation over its cut families and its strengthenings. The naming is as follows: `BBC-LP` uses the dual optimality cuts, `BBC-K` the combinatorial cuts, `+T` adds the triplet bounds, `+F`

enables fractional separation, `+C` the conflict-graph row, `+H` the hybrid-genetic incumbent, `+P` the Pareto-optimal cut lifting, and `+ACC` all three strengthenings together.

The reported quantity. The common metric is the empty-start cost of the sequence a method returns, evaluated by the loading oracle. Each formulation has its own native objective, and two of them use the free-initial convention, so the native values are not comparable across methods. The empty-start cost of the returned sequence is defined for every method regardless of its formulation, and it is what makes the cross-method comparison of Section 6.2 meaningful.

6.2 Validation

Correctness was established before any result was recorded, and again from the results themselves.

Before the campaign. Every exact solver was checked against brute-force optima on a bank of small instances, in every combination of its option flags. The loading oracle itself was verified against an exact dynamic program over magazine states; the verification described in Appendix B repeats this on 349,872 instance-and-sequence pairs and finds no disagreement.

Identifying an instance. An instance is identified by the family it comes from, its file name, its number of jobs, its number of tools, and its magazine capacity. All five are needed. The Crama collection reuses the same file names at four different capacities, so two runs that agree on the name can be runs on different instances. Keying on the name alone merges rows describing different instances and produces apparent disagreements between methods where none exists. The comparison below uses the full identifier.

Cross-method agreement. Of the 1,421 instances, 1,115 were solved to proven optimality by at least one method and 999 by at least two. On those 999 there are 41,673 pairs of methods that both proved an optimum. *In none of them do the two disagree.* Every pair of methods that both closed an instance returned a sequence of the same empty-start cost.

The two conventions, measured. Proposition 5 states that the empty-start and free-initial costs of a sequence differ by the size of the initial load. The campaign confirms it on real instances. The SSPMF model reports the free-initial cost natively, and on all 1,038 instances it closed, the difference between its reported value and the empty-start cost of the same sequence equals the magazine capacity b on 1,029 of them. On the other nine it is smaller than b, and those are exactly the instances on which an optimal sequence

begins with a job that needs fewer than b tools, so the magazine is not filled at the start. The two baselines that report the empty-start cost natively, `CATZ-F4` and LSS, show a difference of zero on all 1,729 runs in which they closed an instance.

Tools that no job needs. Four matrices in the collection contain a tool that appears in no job's requirement set: Crama `s2n007` and `s2n009`, Laporte3 `L28-7` and Laporte5 `L6-8`. Counted with the capacities at which they appear, that is ten of the 1,421 instances. The coverage bound of Corollary 1 must therefore be read as $q = |U| - b$, using the set U of tools that some job actually requires, and not as $|T| - b$ using the tool count declared in the instance file. On those ten the two differ, and the smaller value is the correct one: on Laporte3 instance `L28-7`, which declares 25 tools of which one is never required and has $b = 5$, the optimum in the free-initial convention is 19, which is $|U| - b = 24 - 5$ and not $|T| - b = 20$. All coverage-bound figures in this section use $|U|$.

A defect found and corrected. The Benders master carries the coverage row $\theta \geq |U|$ of Section 5.2 as an explicit constraint. It has to: unlike every other method here, the master has no formulation of the loading problem from which that bound would follow, so without the row it is not available to the relaxation at all. Observation 7 shows how much work the row does — on 87.4% of runs the root relaxation is exactly $|U|$ and nothing else contributes.

Runs that failed. Of the 16,994 runs, 427 returned no result. Forty-nine are a failure of the harness itself while closing its log file on the cluster filesystem, and the remaining 378 are crashes of the worker subprocess. They are not spread evenly: 352 of the crashes belong to the LSS baseline, 239 of those to the Laporte5 family.

The two groups have different causes, and one of them is recorded.

The Benders failures. Of the 75 failures scattered across the nine Benders configurations, at most 26 in any one, 49 carry an explicit exception: `OSError: [Errno 127] Key has expired`. That is expiry of the job's filesystem credentials on a cluster whose home directories are served over an authenticated mount. The run lost the ability to write its own output; nothing about the instance or the solver is implicated. These are infrastructure losses, and they change no comparison drawn between configurations.

The LSS crashes. The remaining 352 are recorded as `subprocess crash` with an empty message: the worker was terminated without raising, so no exception was captured and the proximate cause is not in the result files. What the timing shows is that they are not startup failures and not a bad node. No crash occurs before 1,063 seconds; the median is 2,069 and the maximum 3,582, just inside the limit. They are distributed evenly over the ten shards, at between 32 and 38 each, so no single node or batch accounts for them. And 239 of the 352 are Laporte5.

Configuration	Catanzaro	Crama	Laporte3	Laporte4	Laporte5	Laporte7	All
SSPMF	73/171	68/160	340/340	330/330	161/336	66/80	1038/1417
CATZ-F4	55/171	44/160	340/340	330/330	72/327	44/78	885/1406
LSS	52/132	42/124	340/340	330/330	39/101	41/42	844/1069
BBC-LP+T	38/168	34/156	340/340	192/330	84/336	39/80	727/1410
BBC-LP	38/169	35/151	340/340	191/329	83/338	39/80	726/1407
BBC-K	39/166	34/145	340/340	190/329	83/335	39/80	725/1395
BBC-LP+F+H	43/171	37/160	316/340	157/330	97/339	40/80	690/1420
BBC-LP+F	37/169	32/160	313/340	156/329	85/340	36/80	659/1418
BBC-LP+F+C	37/170	32/160	311/340	157/329	84/336	36/79	657/1414
BBC-LP+ACC	43/170	37/160	286/340	154/330	97/339	39/80	656/1419
BBC-K+F	37/171	21/121	316/340	156/329	86/338	35/79	651/1378
BBC-LP+F+P	35/171	32/160	282/340	155/328	83/336	35/79	622/1414

Table 6: Instances solved to proven optimality, against runs that returned a result, under a one-hour limit. Denominators below the family size are the failed runs of Section 6.2; the LSS row is the one where this matters, and its totals should not be read against the others.

A run that survives at least eighteen minutes and is then killed without raising is consistent with a resource limit reached during the search rather than a defect in the model, but the campaign does not record enough to establish that: it would take the scheduler's accounting record for the job, which is not part of the result files. The obvious size explanation is separately excluded — measured by $n|T|$ the crashed runs are *smaller* than the completed ones, median 400 against 500.

What is established either way is that the loss is not random with respect to difficulty. On the instances LSS completed, SSPMF proves optimality 78% of the time; on the 352 it lost, only 60%. The instances LSS never attempted are the harder ones, so its totals are flattering and its rate is not comparable with the others. Section 7.4 records what follows from that.

6.3 How far each method gets

The question. How many instances does each method close, and does the decomposition compete with the compact models it was built to improve on?

This is the coarsest measurement in the section and the least explanatory. It establishes a ranking, and that ranking is the starting point for everything after it, but by itself it says nothing about *why* the ranking is what it is. Sections 6.4 and 6.5 take that up.

Table 6 reports, for each configuration and each family, the number of instances solved to proven optimality against the number of runs that returned a result.

Three things can be read from Table 6.

The compact models dominate the decomposition. SSPMF closes 1,038 instances against 727 for the best Benders configuration, and that gap is far larger than anything the missing runs could account for.

Among the compact models themselves the ranking is less clear than the totals suggest, because LSS did not complete a third of its runs. On the 1,067 instances that LSS and

SSPMF both attempted, LSS proves 844 optima and SSPMF 827; on the 1,066 that LSS and `CATZ-F4` both attempted, `CATZ-F4` proves 865 against LSS's 844. Since the runs LSS lost are the harder ones, its position in both comparisons is flattered by an unknown amount. The honest reading is that the three compact models are close to one another on this collection and that separating them would need the lost runs recovered; what the campaign does establish is that all three are ahead of the methods built here.

The families separate the methods rather than rank them uniformly. Laporte3 is solved completely by six of the twelve configurations. Laporte4 splits them sharply: the three compact models solve all 330, and the Benders solver solves between 154 and 192. Catanzaro and Crama are hard for everything, and no method closes more than half of either.

Among the Benders variants, the three that do not separate at fractional points (`BBC-LP`, `BBC-LP+T`, `BBC-K`) are the three best, within two instances of one another. Every configuration carrying `+F` is worse than all three. Section 6.4 takes that apart.

Solving times. Table 7 reports times on the 535 instances that all eight of the listed configurations solve. The shifted geometric mean with a shift of ten seconds is used, following standard practice for exact-method comparisons, because it is insensitive to both very small and very large individual values.

Method	shifted geometric mean (s)	median (s)	maximum (s)
SSPMF	1.90	0.140	48.7
`BBC-K`	14.50	0.018	845.8
`BBC-LP+T`	16.03	0.066	1126.0
`BBC-LP`	16.11	0.067	1128.4
`BBC-LP+F+H`	30.74	0.003	3584.6
`BBC-LP+F`	33.73	0.107	3586.6
LSS	37.68	18.254	2211.5
`CATZ-F4`	92.83	74.304	3506.5

Table 7: Solving times on the 535 instances that all eight configurations solve. The Benders solver has by far the lowest median and a high mean, which is the signature of a method that either finishes almost at once or struggles.

The pattern in Table 7 is as informative as the ranking. The Benders solver's median is one to three orders of magnitude below every compact model's, while its mean is an order of magnitude above SSPMF's. It either closes an instance immediately or does not close it at all. Section 6.5 identifies what separates the two cases, and it is not the size of the instance.

How the solved count accumulates. The shifted geometric mean summarises the instances a method solves; it says nothing about the ones it does not. Table 8 gives the

whole curve: how many instances each method has closed by one second, ten seconds, and so on to the limit.

Method	1s	10s	60s	300s	900s	3600s	attempted
SSPMF	482	735	946	959	982	1038	1417
`CATZ-F4`	128	254	444	662	795	885	1406
LSS	57	246	532	759	819	844	1069
`BBC-LP`	403	469	525	572	650	726	1407
`BBC-K`	431	490	531	570	699	725	1395
`BBC-LP+F`	362	435	466	508	542	659	1418

Table 8: Instances closed within a given time. The two groups have different shapes.

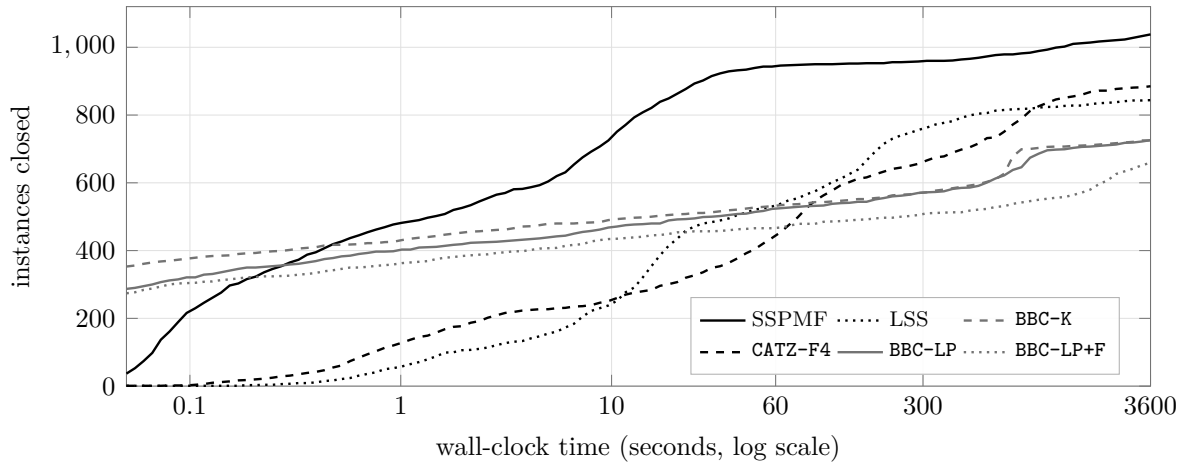

Figure 10: Every solved run in the campaign, not the six sampled columns of Table 8: for each method, the number of instances closed by each wall-clock time. The curves cross, and where they cross is what matters here. Before about two seconds the configurations built here lead the field — `BBC-K` has a median solve time of 0.06 seconds against 59.8 for `CATZ-F4` — because a Benders master with a tight coverage row closes an instance at the root immediately or not at all. After two seconds they stop moving and every compact model overtakes them. The decomposition is not running out of time. It has stopped making progress, and what would restart it is a bound that improves when the solver branches.

The shapes differ, and the difference is the point. `BBC-LP` closes 403 instances in the first second and 726 in the full hour: more than half of everything it will ever solve is solved before a compact model has finished building its model. After that it flattens. SSPMF starts lower relative to its own total, at 482 of 1,038, and keeps climbing throughout; LSS closes 57 in the first second and 844 by the end. Extending the limit would therefore not affect the methods equally. The Benders solver is not slow. It is fast on one class of instance and unable to finish on the other, which is the behaviour Section 6.5 explains.

How far from proven the unsolved runs are. The complement of the solved count is the gap remaining when the limit is reached, measured between the incumbent and the

dual bound. For `BBC-LP` the median over its 681 time-limited runs is 22.9% and the mean is 28.6%, with a maximum of 73%; the nine Benders configurations lie between 26.0% and 29.5% on the mean. Only 50 of those 681 runs finish within five per cent of proven.

That figure should be read against the one in the next paragraph. On the runs where the optimum is known independently, the solver's incumbent already equals it 59% of the time. It is therefore sitting on the right answer with a certified gap of roughly a quarter, and the hour is spent failing to close a gap that exists only in the certificate.

6.4 The Benders solver and its strengthenings

The question. A branch-and-bound method can fail for two reasons: it cannot find a good solution, or it finds one and cannot prove it optimal. The two call for opposite remedies, and the ablation below is designed to tell them apart. Four strengthenings were implemented, one aimed at the incumbent and three at the dual bound. If the difficulty is search, the first should be marginal and the others should help. If the difficulty is the bound, the pattern should be reversed.

That is what makes this subsection informative rather than merely a list of settings: the outcome discriminates between two explanations, and it was not obvious in advance which one it would pick.

Fractional cuts. Separating Benders cuts at fractional points is a standard acceleration whose value on this problem had not been established. The measurement below settles it.

The cuts are now generated in very large numbers. Across the 1,415 instances that both `BBC-LP` and `BBC-LP+F` attempted, the fractional configuration added 67,513,396 cuts, and the configuration without fractional separation added none, which is the expected control.

They do not help. Table 9 gives the comparison.

On the 1,415 common instances	`BBC-LP`	`BBC-LP+F`
fractional cuts added	0	67,513,396
solved to optimality	726	659
median nodes explored	77,504	3,795
final dual bound, on the 1,404 instances where both are recorded:		
strictly higher with fractional cuts		0
equal		1,228
strictly lower with fractional cuts		176

Table 9: The effect of enabling fractional Benders cuts. Sixty-seven million cuts enter the master. The bound at termination is never higher than without them, and on 176 instances it is lower. The node counts are consistent with the hour going into generating and managing cuts instead of searching, though the campaign does not isolate that mechanism. Sixty-seven instances that the base solver closes are lost.

The mechanism works: the cuts are valid, they are generated, and they enter the master. The node counts show they do tighten the relaxation locally, since the search explores twenty times fewer nodes. What they do not do is raise the bound that matters. On no instance in the comparison is the bound at termination higher with them than without.

The reading is that the fractional dual information is not new information. The integer cuts already give the master everything the loading dual has to say, and the fractional ones re-derive it at a cost of an hour of separation.

The three strengthenings. Table 10 compares each strengthening against `BBC-LP+F` on the instances both attempted.

Configuration	what it adds	common	solved	difference
`BBC-LP+F`	—	—	—	—
`BBC-LP+F+H`	hybrid-genetic incumbent	1,421	690	+31
`BBC-LP+F+C`	conflict-graph row	1,421	657	−2
`BBC-LP+ACC`	all three together	1,421	656	−3
`BBC-LP+F+P`	Pareto-optimal cut lifting	1,421	622	−37
`BBC-LP`	fractional separation removed	1,415	726	+67

Table 10: Each strengthening against `BBC-LP+F`, on the instances both attempted. Only the one that improves the incumbent helps. The two that aim at the dual bound cost more time than they return, and combining all three does not recover the loss. The last row is the comparison of Table 9, repeated here because removing a feature outperforms every addition.

The pattern is consistent and it is the same pattern as the fractional cuts. The only strengthening that helps is the one that supplies a better *incumbent*. Every attempt to improve the *bound* costs more than it returns. The conflict-graph row is neutral, which is what Section 5.3 predicted for it on structural grounds: on most instances the coverage row already dominates it.

Where the search stops. Two further measurements locate the difficulty precisely.

First, of the 726 instances `BBC-LP` solves, 259 are closed at the root node, without branching at all. The root relaxation value was recorded on 1,178 of the 1,421 runs, and on 883 of those the optimum is also known. On those 883 the root value equals the optimum 294 times, and the split is sharp: among the runs that went on to close the instance it is already the optimum in 264 of 497, and among the runs that did not close it in only 30 of 386. Where it is not the optimum it is short of it by at least one unit and by 16.7% at the median, rising to 42.9%.

Second, consider the runs that hit the time limit. Across the nine Benders configurations there are 6,562 of them, and for 3,874 the optimum is known because some method proved

it. On 2,571 of those 3,874, which is 66%, the Benders solver's incumbent already *equals* that optimum. It had found the answer and could not prove it. For `BBC-LP` alone the figure is 229 of 386, or 59%.

Taken together these two measurements describe a solver that finishes at the root when the bound is good, and that finds the optimum quickly and then spends an hour failing to certify it when the bound is bad. In neither case is the search the constraint on what it can do.

6.5 The coverage bound decides the outcome

Section 4 predicted that the coverage bound q would separate easy instances from hard ones for any configuration-based method. That prediction is directly testable, because for each instance with a known optimum it can be checked whether the optimum equals the bound.

Of the 1,115 instances whose optimum is known, 523 have the coverage bound tight and 592 do not. The split is close to even, at 46.9% tight, which means the standard instance collections are not concentrated in either regime. Where the bound is loose it is loose by a wide margin: the optimum exceeds it by 5.71 on average and by as much as 30.

Splitting the results by that criterion gives Table 11.

	instances	`BBC-LP` solves	SSPMF solves
coverage bound tight ($Z^* = q$)	523	492 (94%)	523 (100%)
coverage bound loose ($Z^* > q$)	592	234 (40%)	515 (87%)

Table 11: Success rate split by whether the coverage bound equals the optimum. The bound, not the size of the instance, is what decides the outcome for the Benders solver. The multicommodity model is affected by the same split but far less. A natural reading is that a compact model gives its search a full description of the problem at every node, so the search recovers what the bound does not supply; the campaign shows the difference but does not test that explanation.

This is the central measurement of the campaign. The same solver, on the same hardware, under the same time limit, closes 94% of one class and 40% of the other. The classification is not by number of jobs, number of tools or density; it is by a single structural property of the instance, and it is the property the theory of Section 4 identifies.

The SSPMF column is the useful control. Its relaxation is also equal to q, so if the bound were the whole story it should show the same collapse. It does not: it falls from 100% to 87% where the Benders solver falls from 94% to 40%. A compact model gives the branch-and-bound search a full description of the problem to work with at every node, and the search recovers much of what the bound does not supply. The Benders master sees only the cuts it has accumulated, and when the bound is loose it has very little else.

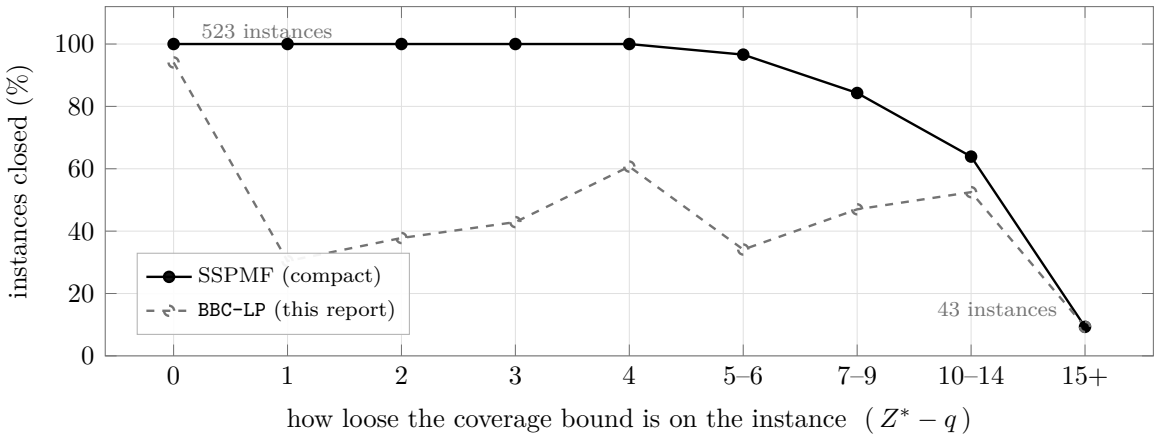

Figure 11: The 1,115 instances of known optimum, grouped by how far the optimum exceeds the coverage bound, against the share each method closes. This is Table 11 resolved into its whole response curve, and the curve says more than the two-way split does. The Benders solver does not degrade: it falls off a cliff between a bound that is exact and a bound that is wrong by one, from 94% to 30%, and then wanders without trend — once the bound is loose at all, by how much stops mattering. The compact model degrades smoothly instead, holding 100% out to a gap of four. But the two meet again at the right-hand end: on the 43 instances whose bound is loose by fifteen or more, *both* close 9.3%. The compact model is not immune to the bound; it has further to fall.

6.6 Branch-and-price

The question. Everything above measures the bound indirectly, through how many instances a solver closes. The prototypes can answer directly, because they report the value of their root relaxation on every run. Two propositions of Section 5.4 predict what that value will be — that PCF′ equals the coverage bound and that PTF can exceed it — and the campaign is a chance to test a proof against a benchmark rather than against another proof. That is the purpose of this subsection; the solve counts are secondary and, for the reasons given next, weak evidence.

The two position-indexed prototypes were run separately from the campaign above. They are implemented in Python over PySCIPOpt with a Python pricer, so a single node costs orders of magnitude more than a node of a compiled solver, and comparing their wall-clock times against CPLEX would measure the implementation language rather than the formulation. In total 2,581 runs were made over 469 instances in seven configurations: PCF′ alone; PCF′ with each of the four accelerations of Section 5.4 separately, written `+HP` for heuristic pricing, `+MC` for multiple pricing, `+WS` for warm starting and `+STAB` for dual stabilisation; PCF′ with all four together, written `+ACC`; and PTF. The time limit is thirty minutes on the Catanzaro and Crama families and ten minutes on the Laporte families.

Two properties of this run make its counts weaker evidence than the campaign above, and both are stated before the numbers. First, the harness carries an early-stop rule:

once a configuration has failed to close six increasingly hard instances in succession, the remaining harder instances are skipped for that configuration. The number of runs a configuration received therefore depends on how it performed, so the denominators in Table 12 are not comparable across rows. Second, no repetition or seed variation was run. Every comparison drawn below is made either on instances that both configurations attempted, or on the root relaxation value, which the early-stop rule cannot bias because it is recorded before any search.

They agree with the campaign. On the 507 runs that proved an optimum and where the same instance was also closed in the campaign of Section 6.3, the two agree on every one. As in Section 6.2, the native objective is the free-initial cost and the difference from the empty-start cost is exactly b on all 507.

What they solve. Table 12 gives the counts. Nothing above nine jobs was closed by any configuration: all 608 runs on the 30-job and 40-job instances reached the time limit. This is the expected consequence of a Python pricer and is not evidence about the formulation.

Configuration	what it adds	runs	solved	median time (s)	median columns
PCF′`+MC`	multiple pricing	422	102	11.4	1,504
PCF′	—	415	91	23.7	1,574
PCF′`+HP`	heuristic pricing	360	73	19.7	5,173
PTF	position–transition rows	387	73	2.4	1,279
PCF′`+WS`	warm-started pool	359	64	14.9	1,610
PCF′`+ACC`	all four together	337	60	28.1	1,221
PCF′`+STAB`	dual stabilisation	301	44	35.7	1,133

Table 12: The branch-and-price prototypes. Median time is over the runs that closed; median columns is over all runs. The run counts are outcome-dependent through the early-stop rule, so the rows are not comparable; the comparison that matters is the one below, which is made on common instances.

What the root relaxation shows. Proposition 20 states that the position–configuration relaxation equals the coverage bound q, and Proposition 22 states that the position–transition relaxation can exceed it. Both are testable directly, because the prototypes record the root relaxation value.

PCF′ never exceeds q. On all 2,114 PCF′ runs for which a root relaxation value was recorded, that value equals $|U| - b$ exactly. Not one run, in any of the six PCF′ configurations, produced a root bound above the counting bound. This is Proposition 20 confirmed on the benchmark rather than on paper.

PTF exceeds q, and almost never. Of the 233 PTF runs with a recorded root relaxation, three are strictly above $|U| - b$: Catanzaro `A0-0`, where the bound is 2.013 against $q = 2$,

and Laporte3 `L11-6` and `L11-7`, where it is 6 against $q = 5$. Proposition 22 is therefore not vacuous. But 230 of 233 runs sit exactly on q, and on the two Laporte3 instances the optima are 12 and 13, so a bound of 6 instead of 5 closes a small part of a large gap.

The bound is often already the answer. On 990 of the 1,846 runs where both the root relaxation and the optimum are known, the root relaxation *equals* the optimum. These are the tight instances of Section 6.5, seen from the other side: where the coverage bound is tight, a configuration-based relaxation needs no strengthening at all, and where it is loose, neither PCF′ nor PTF supplies one.

Head to head on the 320 instances both attempted, PCF′ and PTF each close 73. Their node counts differ by a factor of fifty — a median of 96 against 2 — because the PTF pricer prices coupled pairs of configurations and costs far more per node. The stronger relaxation does not convert into more instances closed.

The comparison with the Job Grouping Problem. Otiai [38] builds a branch-and-price for that problem whose set-covering relaxation is strong enough to close benchmark instances in a handful of nodes. For the SSP, Moreira [35] reports that the trivial bound $|T| - b$ obtained by the multicommodity model was the best available bound on all thirty instances of three of his subgroups, with the configuration model giving the best bound on four instances of the remaining, tighter subgroup. The measurements above say the same thing on a larger collection and with the mechanism visible: the configuration relaxation for the SSP is not, in general, an improvement on the counting bound.

6.7 What the campaign establishes

Two of the results above are checks that the study is sound. Cross-method agreement is complete: on the 999 instances solved by more than one method, all 41,673 pairs agree on the cost of the schedule returned. And the three reimplemented compact models all close substantially more instances than any configuration built here, without being cleanly separable from one another, because the LSS runs that failed are the harder ones.

The remaining four converge on one statement. Whether the coverage bound equals the optimum decides whether the Benders solver closes an instance, and where it fails the incumbent is usually already optimal. Of four strengthenings, only the one that improves the incumbent helps; the three aimed at the bound are neutral or harmful, and the sixty-seven million fractional cuts are the extreme case, raising the bound at termination on no instance while costing sixty-seven solved ones. Both branch-and-price relaxations sit on the coverage bound or barely above it, confirming Propositions 20 and 22 and showing the second to be of little practical use as it stands. Section 7 takes that up.

7 Discussion

7.1 Bound-limited, not implementation-limited

A method can fail to solve an instance for two reasons. It can fail to find a good solution, or it can find one and fail to prove that no better one exists. The distinction matters because the two failures call for different remedies. The first is answered by better search: heuristics, warm starts, branching rules. The second is answered only by a better bound.

For the SSP four independent measurements separate the two cases, and all four point the same way.

The incumbent at the time limit. On the runs that hit the limit and whose optimum is known from elsewhere, the Benders solver's incumbent already equals that optimum 66% of the time across its nine configurations, and 59% of the time for the plain one. In those runs the solver has found the answer and spends the remaining hour failing to certify it.

The ablation. Four strengthenings were implemented. The one that improves the incumbent, a hybrid genetic search run at the root, gains thirty-one instances. The three that aim at the dual bound — fractional cuts, Pareto-optimal lifting, a conflict-graph row — are neutral or harmful, and the worst of them costs thirty-seven. A solver whose difficulty was finding good solutions would not behave this way.

The split by bound tightness. The same solver closes 94% of the instances on which the elementary bound q happens to equal the optimum, and 40% of those on which it does not. The classification is by a single structural property rather than by size or density, and it is the property Section 4 identifies as governing the problem.

The other method. The branch-and-price prototype reports its root relaxation on every run, and on all 2,114 runs of the position–configuration formulation that value is exactly q. A different algorithm, a different formulation and a different solver arrive at the same number.

The natural objection is that this is a statement about one implementation. It is not, and Proposition 20 is why: the position–configuration relaxation, arrived at from a completely different direction, equals q exactly. So does the multicommodity relaxation of da Silva et al. [16], proved by its own authors. Three independent routes reach the same value. A configuration-based exact method for the SSP is therefore limited by the strength of its bound and not by its implementation.

The fractional-cut result is the strongest form of this conclusion. Separating at fractional points is the standard remedy when a Benders scheme tails off, and it does what it is supposed to do locally: sixty-seven million cuts reduce the node count by a factor of twenty. It nonetheless raises the bound at termination on no instance at all. The remedy is being applied correctly, to the wrong obstacle.

7.2 Where a stronger bound could come from

Section 7.1 establishes that the methods of this report are limited by their bound. That is a diagnosis of the symptom. This subsection asks what causes it, and measures what a different family of inequalities could be worth. The cause is structural rather than incidental, which is consistent with an ablation in which none of the dual-bound strengthenings helped.

The cost is not a local quantity. Write β_t for the number of maximal blocks of consecutive positions during which tool t sits in the magazine. Every insertion of t begins one such block, so

$$Z^* \;=\; \sum_{t \in U} \beta_t, \qquad \text{and} \qquad \beta_t \geq 1 \;\text{ for every used tool}$$

recovers the coverage bound exactly. Everything above q is therefore a *re-insertion*: a tool evicted and brought back. A re-insertion happens when the jobs needing a tool are pushed apart by other jobs whose requirements cannot sit alongside it. That is a property of the global interleaving of the schedule. It is not a property of which pairs of jobs happen to be adjacent.

Every inequality in this report is local. Against that, consider what the two methods actually give their relaxations.

The Benders master carries one row $\theta \geq \sum_{ij} w_{ij} x_{ij}$, which is indexed by arcs; the triplet rows $\theta \geq w_{ijk}(x_{ij} + x_{jk} - 1)$, which are the weak linearisation of a product and are worth nothing at a fractional point where both arcs sit at one half; and the constant coverage row. Every optimality cut it generates is also arc-supported, with coefficient $\sum_t \lambda_{ijt}$ on x_{ij}. So the master learns only adjacency-indexed facts about a cost that is not adjacency-indexed.

The position–configuration relaxation prices one configuration per position and couples positions only through per-tool presence rows, so its linear relaxation mixes configurations fractionally at each position and pays no transition. That is why Proposition 20 holds, and it is why the measured root value is q on every one of 2,114 runs.

PTF is the one construction here that ever exceeds q, and the reason is the same one stated positively: a column carries a transition, so the relaxation cannot avoid paying for it. Why it exceeds q so rarely is not established. The candidate explanation is that its chaining between consecutive pairs is enforced tool by tool rather than configuration by configuration, which would restore the fractional mixing one level up; that is a conjecture about the formulation, and testing it means building the stronger chaining and measuring.

Observation 7 (The coverage row is carrying the master alone)**.** On 9,084 of the 10,396 Benders runs for which a root relaxation value was recorded — 87.4% — that value equals $|U|$ exactly. The pairwise row and the triplet rows contribute nothing at the root on almost nine runs in ten. Where the pairwise structure does bite it bites hard, by 7.70 on average, but that is confined to the dense instances.

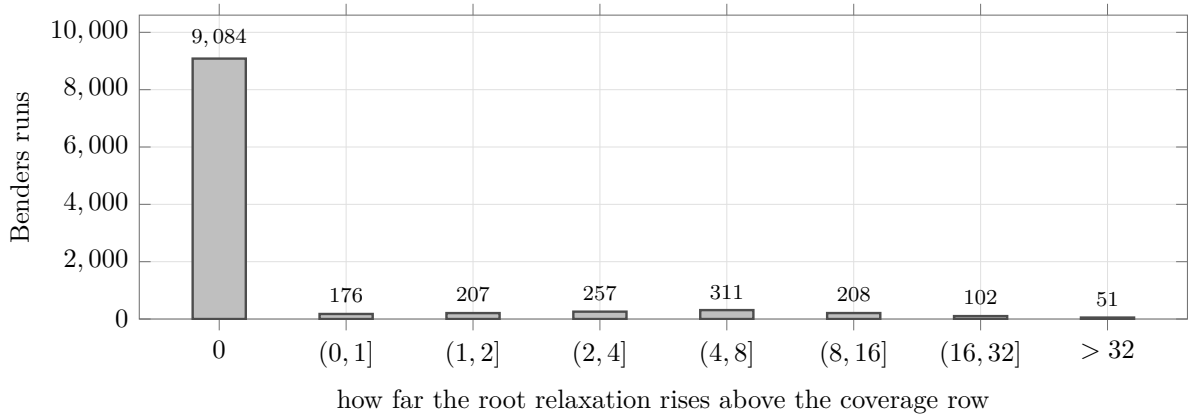

Figure 12: Observation 7 drawn over all 10,396 Benders runs that recorded a root relaxation. The first bar is every run whose root value is *exactly* the coverage row — 9,084 of them, more than seven times the whole remainder combined. The pairwise row, the triplet rows and every optimality cut generated before the first branching decision, taken together, move the relaxation not at all on eighty-seven runs in a hundred. This is the diagnosis behind the rest of this section stated as a single measurement. The master has no shortage of cuts; it is carrying one constant row and getting nothing from the others.

A family that is not local. Let $W = [p, r]$ be a contiguous block of positions with $p \geq 2$. Write $w_{t,k}$ for the insertion of tool t at position k and $a_{t,k}$ for its presence, as in the position-indexed formulation of Section 5.4.

Proposition 23 (Window inequality)**.** *For every contiguous block of positions $W = [p, r]$ and every feasible point of PCF,*

$$\sum_{t \in U} \sum_{k=p}^{r} w_{t,k} \;\geq\; \Big|\{t : a_{t,k} = 1 \text{ for some } k \in W\}\Big| \;-\; \sum_{t \in U} a_{t,p-1}. \tag{1}$$

Proof. Let S be the set of tools present at some position of W and A the set present at position $p-1$. Fix $t \in S \setminus A$. Then $a_{t,p-1} = 0$ and $a_{t,k} = 1$ for some $k \in W$, so there is a smallest such k, and at it $a_{t,k} - a_{t,k-1} = 1$. Constraint (W) gives $w_{t,k} \geq 1$. Distinct tools contribute to distinct terms, so the left-hand side is at least $|S \setminus A| \geq |S| - |A|$, which is the right-hand side. □

The inequality linearises with one auxiliary variable per tool and window. Two properties make the family useful. It constrains a stretch of the schedule rather than a

single point, so it can charge a re-insertion that no arc-indexed or single-position row can see. And it contains only master variables, so it is a *robust* cut in the sense of Poggi and Uchoa [41]: its dual never reaches the pricing problem, and the set-union knapsack oracle of Section 5.4 is unchanged. The per-tool counting rows (T) already in the master are the case $W = [2, n]$ summed over tools, so Proposition 23 generalises them.

The implementation was additionally checked against the loading rule at 1,200,816 integer points — every window of every sequence of a bank of random instances — with no violation. That is a test of the code, not of the proposition.

What it would be worth. A family is worth implementing only if the bound it could deliver is materially better than the bound already available. That can be settled before writing any solver code, by computing for each family the best bound it could possibly give and comparing it against the optimum. On an instance small enough to enumerate, the ceiling of a family is the minimum over sequences of the strongest bound that family certifies for that sequence: for the arc-supported family this is the pairwise weight solved to optimality as a Hamiltonian path, and for the window family it is the best decomposition of the sequence into disjoint windows, an $O(n^2)$ dynamic program. Table 13 reports both on the 62 instances of the Laporte3 family whose coverage bound is loose.

Family	mean	median	best case	no gain on
arc-supported (what the solvers carry today)	8.9%	0%	60.0%	46/62
window	30.9%	40%	66.7%	18/62

Table 13: The fraction of the gap $Z^* - q$ that each family could close at best, on the 62 Laporte3 instances with $n = 8$ whose coverage bound is loose. In absolute terms the arc family accounts for 54 of the 305 units of gap and the window family for 116. Both figures are ceilings: a linear relaxation lands below them. The arc ceiling is the one that matters, because it says the families now in the solvers have almost no headroom left however they are tuned.

Two readings follow. The arc-supported family is exhausted: on three quarters of the loose instances the best bound it could ever certify is the coverage bound itself. And the window family has real headroom, roughly two fifths of the remaining gap at the median. Measured on the same instances, 84% of the windows that carry the bound have length at most four and 71% have length two or three, so a static pool of short windows captures most of the value and no separation routine is required.

Observation 8 (The window family is inactive at the PCF′ root)**.** The family of Proposition 23 was added to the PCF′ master — 15 windows of length two to four, 660 rows and 225 auxiliary variables per instance — and the root relaxation recomputed on 37 of the 62 loose Laporte3 instances. The root value equals q on all 37 with the rows and without them; the family raises the bound on none, including instances whose gap is ten

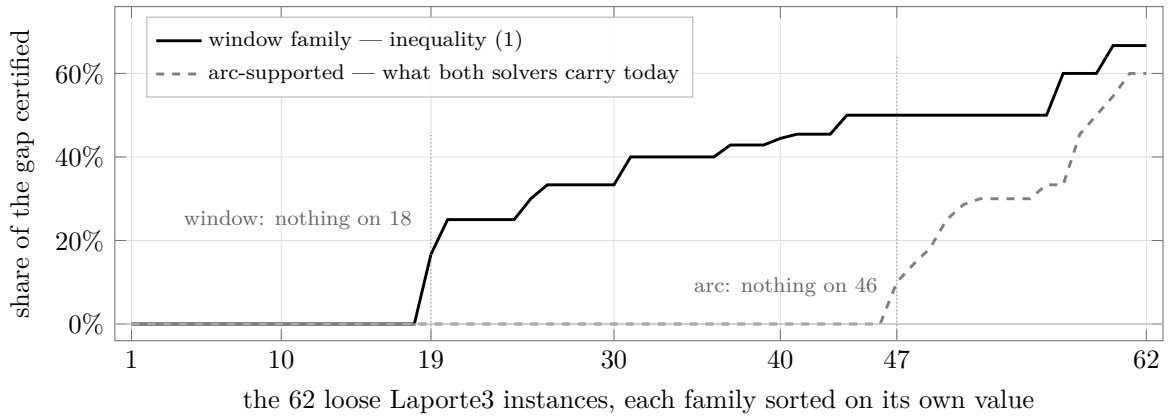

Figure 13: The ceiling of each family on each of the 62 instances, each family sorted independently on its own values, so the two curves answer the question *how often does this family certify anything, and how much*. Each is the fraction of that instance's gap the family could close at best, computed by enumerating every ordering, so both are limits a linear relaxation falls short of. The arc-supported family — all either solver in this report carries — is flat at zero for the first 46 instances and only then rises; on three quarters of these instances the strongest bound it could ever produce is the coverage bound it already has, however it is separated or tuned. The window family is flat at zero for 18 and then climbs to a plateau near two thirds. The two peaks are similar, 60% against 67%, so the difference between the families lies in how often each certifies anything at all, rather than in how much it certifies when it does.

or eleven. Every window row is slack at the LP optimum, by between 0.5 and 3.4. The reason is visible in the solution: the LP holds each tool partially present at every position, so $\sum_t \max_{k \in W} a_{t,k}$ stays near b instead of counting the distinct tools an integral schedule would need inside W, and no tool ever *enters* a window. The relaxation evades the family in the same way it evades the arc-supported ones.

Where the family belongs. In the position-indexed master, not the Benders master. Expressing (1) in arc space needs a family of job sets to be simultaneously contiguous, which is a product of indicators and has no useful linear form — the same obstruction that makes the triplet rows worthless. In the position-indexed master the window is a range of position indices and the inequality is immediate. That is an argument for the branch-and-price side of this report over the Benders side, and it is the reverse of the ranking in Table 6: the method that solves fewer instances today is the one with somewhere left to go.

7.3 Where the theory and the measurements meet

Three agreements between Section 4 and Section 6 are worth drawing out, because each was arrived at independently.

The bound governs both sides. Section 4 shows that q equals the optimum on a large part of the instance space, and that where it does the two-phase heuristic tends to be exact. Section 6 shows that where it does, the exact methods close at the root, and where it does not they branch without closing. The same quantity that makes the heuristic exact makes the exact methods fast. That is not obvious in advance: it says that the instances on which one would most want an exact method are exactly the instances on which the available exact methods are weakest.

The gap opens where the magazine is large. Observation 4 finds that every instance with a positive heuristic gap needs $b \geq 3$, and that positive gaps are rare in a random sample. Independently, Burger et al. [7] report from an industrial study that the decomposition does well when the largest job nearly fills the magazine, and that its solution quality decreases as the gap between the magazine size and the largest job size grows. One of these is an exhaustive combinatorial search on small instances and the other is a case study on a printing press. They describe the same regime.

The walk model overstates the weakness of the standard method. This is the least expected of the three. The literature's mental model of the two-phase heuristic is a walk through one magazine per group, and under that model the canonical example family has a positive gap that grows without bound. Under the method as it is actually specified, with its final loading step, that same family has no gap at all (Observation 2). The standard method is better than the standard picture of it. Problem 1 asks how much better, and that question did not exist before the two readings were separated.

7.4 Limitations

Six, ordered by how much they threaten the conclusions above.

The census of heuristic gaps is on small instances. Observation 4 says that positive heuristic gaps are rare, always equal to one, and never occur at $b = 2$. The census runs from three to eight jobs, because the optimum there is computed by exhaustion. Nothing in it shows the same holds at industrial sizes, and the agreement with Burger et al. [7] comes from a different method rather than a wider range of sizes. An earlier and narrower version of this census supported a stronger claim, that positive gaps need $b \geq 4$, which the extended search refuted (Remark 3); the claim now made is correspondingly weaker. This remains the least secure empirical statement in the report and the first a reader should discount. The structural results of Sections 4.1 and 4.2 do not depend on it; those are proved for every instance and every capacity.

A ceiling is not a bound, and for the window family the difference is everything. Table 13 and Figure 13 give the strongest bound each family could certify, found by enumerating every sequence of 62 instances at eight jobs. Both families are measured the same way, so the comparison between them is fair, and the arc-supported family being exhausted is a claim about a ceiling and therefore safe. The window family's ceiling, however, turns out to be unreachable. Added to the PCF′ master and measured at the root on 37 of those instances, the family raises the relaxation on *none* of them (Observation 8). The ceiling was computed over integral sequences; the relaxation is fractional, and the two are not comparable. Any future ceiling computation in this line should be read with that in mind.

Four hundred and twenty-seven runs returned no result. This affects the denominators in Table 6 and is the one failure that changes a conclusion. Of the 352 belonging to LSS, each is recorded as a subprocess termination with no captured exception, so the proximate cause is not in the result files; none occurs before 1,063 seconds, they are spread evenly over the ten shards, and the crashed runs are smaller than the completed ones, which rules out the obvious size explanation. The loss is not random with respect to difficulty either: SSPMF proves optimality on 78% of the instances LSS completed and on 60% of those it lost, so LSS never attempted the hard third of the collection. That is why Section 6.3 reports the three compact models without ranking them against each other. It does not touch the comparison between the compact models as a group and the methods built here, which is an order of magnitude larger than the missing runs. Of the other 75, spread across nine configurations, 49 carry a credential expiry error from the cluster filesystem and are infrastructure losses.

The comparison is one hour, one machine, one collection, and F4 rather than F5. Every solve count and time in Section 6 comes from a single one-hour limit on one homogeneous node type over the standard collections; Figure 10 shows how the counts would move under a longer limit and that they would not move equally for all methods. Catanzaro et al. [9] recommend Formulation 5, which was not implemented, so the compact baseline here is weaker than the best available. Both points run against this report's own conclusion rather than for it, and neither touches the bound results, which concern relaxation values rather than solver speed.

The branch-and-price prototypes run in Python; the baselines run in C. A node of a Python pricer costs orders of magnitude more than a node inside CPLEX, and the counts in Table 12 are additionally outcome-dependent through the early-stop rule. Nothing above nine jobs was closed, and that carries no information about the formulations. None of the conclusions drawn from these runs uses a solve count: they

rest on the root relaxation value, recorded before any search and so affected by neither problem.

The learned cut-selection prototype was not run on the SSP. It was trained and measured on knapsack cover cuts, with one seed per size and no repetition. It shows that the agent learns, and says nothing about this problem. That bears on Section 5.5 and nothing else, and Section 7.2 explains why running it here would not have helped: cut selection chooses among cuts a method can already generate, and those cuts have almost no headroom left.

8 Conclusions and further work

8.1 What this report establishes

The equivalence between the SSP and a walk through magazine configurations organises everything above. It yields a structural theory of the standard heuristic and a family of exact methods, and one quantity — the coverage bound — governs both.

On the structural side. The two-phase heuristic admits two readings, and separating them is a prerequisite for saying anything correct about it (Section 2.5). Within that setting:

- The optimum equals the cheapest walk over *all* feasible groupings (Theorem 1), so the heuristic's loss is exactly the price of using the fewest groups (Corollary 2).
- Zero-gap classes exist for every capacity: two groups, or a tool set barely exceeding the magazine, or a small optimum (Corollaries 3 and 4).
- The ratio is bounded for every instance by $\min(b, K^* - 1, q)$ (Theorem 2), which tightens automatically on instances with few groups or a small tool excess.
- At three groups the walk cost has a closed form (Proposition 6), and the gap obeys a counting bound (Proposition 7) that is attained by explicit witnesses. The natural guess gap $\leq K^* - 2$ is false for the walk value (Proposition 9).
- For the method as actually specified, positive gaps are rare, always equal to one in everything tested, and never occur at $b = 2$ (Observations 2–4). Unboundedness survives, but with a different witness family from the one the walk model suggests (Observation 5).
- The circuit clutter of the grouping system computes K^* (Proposition 12) but does not determine the gap (Proposition 14), while blocking duality gives a first non-integral family for the grouping relaxation (Proposition 16).

- A per-changeover setup cost interpolates between the SSP and the JGP, with the transition confined to an interval whose width is the gap itself (Theorem 3 and Lemma 4). This answers a question posed by Burger et al. [7].

On the algorithmic side. A branch-and-Benders-cut solver was built whose subproblem is exact and polynomial, together with position-indexed formulations carrying a fixed polynomial row set, among which the position–transition model is a configuration relaxation that can provably improve on the coverage bound (Proposition 22), although the campaign finds it doing so on three runs out of 233.

On the empirical side. A campaign over 1,421 instances and twelve configurations, under a single one-hour limit, is reported in Section 6 and summarised in Section 6.7. Four of its measurements — the incumbent at the time limit, the ablation, the split by whether the coverage bound is tight, and the root relaxation reported by the branch-and-price prototypes — are independent of one another and agree. What they agree on is that the obstacle is the bound rather than the search: nothing that improves the search closes the loose instances, and nothing built here improves the bound. Two further measurements exist to make that reading trustworthy rather than to support it, and both do: no two methods that proved the same optimum disagree on its value, and the reimplemented compact baselines are reported without a ranking, because the runs that failed are the ones that would have decided it.

The methods built here are therefore bound-limited rather than implementation-limited: a negative result about a family of methods rather than about one solver. Section 7.2 sets out where a stronger bound could come from.

8.2 Further work

The exact worst case of the heuristic. Section 4.3 settles the extremal behaviour of the walk value at three groups. For the value the method returns, the question is open and is now the more interesting one, because Section 4.4 shows that the two differ substantially. Two concrete questions: is the heuristic gap ever larger than one on a connected instance, and does gap $\leq K^* - 2$ hold for it? Directed searches across $b \in \{3, 4, 5\}$ found no counterexample to either. Problem 1 is the structural question underneath both.

The polyhedral questions. One of the two questions from Section 4.5 is now answered. Observation 6 enumerates the whole $b = 3$ family with two tools per job on at most six tools: half of it has a non-integral covering relaxation, eleven of those instances are bipartite, and the smallest obstruction is a three-job star rather than a ring. What the star and the odd ring share is odd structure in the compatibility hypergraph on *jobs*,

not circular structure among the tools, and characterising exactly which such structures obstruct integrality is the question that replaces the one that has been closed. The second question is unchanged: which facets of the position–transition polytope explain the three runs where its bound exceeds q.

A bound that reaches into the loose half. This is the direction the campaign points to most strongly, and Section 7.2 takes the first step along it: the arc-supported information the solvers carry today is exhausted, and the window family of inequality (1) has roughly four times the remaining headroom at its ceiling (Table 13). That family has now been measured inside the master and it delivers nothing: the root relaxation is unchanged on all 37 instances tested (Observation 8). The robust encoding is the reason. Counting the tools an integral schedule would need inside a window requires tying the row to the column variables, which puts the cut's dual into the pricing problem and forfeits the one property that made it cheap. Whether a non-robust encoding pays for itself is open, and so is whether the rows help inside the tree despite leaving the root alone. Two further families suggest themselves from the same reading: chaining PTF per configuration rather than per tool, which is what confines its advantage to three runs in Section 6.6, and disaggregating the Benders master's single θ into one variable per position, which is the standard strengthening the ablation of Section 6.4 never tested.

Making PTF's advantage large. Section 6.6 finds the position–transition relaxation above the coverage bound on three runs of 233, by one unit each. The construction is therefore sound and, as it stands, of no practical use. Problem 2 asks whether the excess can be made to grow; the polyhedral question underneath it is which facets of the position–transition polytope are responsible for the three cases where it does exceed q. Independently, the prototypes are written in Python with a Python pricer, and a compiled pricer would allow the formulations to be measured on instances larger than nine jobs, which is where they currently stop.

Non-uniform tool costs. Section 5.2 notes that the Benders subproblem is the loading linear program rather than the KTNS rule, and that Crama et al. [14] show the linear program remains exact when tools have different setup times while the rule does not. The solver therefore extends to that regime unchanged. Testing it there would broaden the scope of the method beyond the uniform problem that the whole SSP literature assumes.

Beyond the SSP. Nothing in the position–transition construction is specific to tool switching except the transition metric. Position-transition columns with per-element consistency rows apply to any generalised travelling-salesman problem with overlapping

clusters [42]. Whether the relaxation strength observed here persists in that generality is open.

Learning-augmented methods. Machine learning for the exact side is an active area, covering cut selection [18, 46] and column and branching selection [6, 20, 34]. Its established role is to *choose* among cuts or columns one already knows how to generate, not to make them stronger. Section 7 bounds how much that can help here: what defeats the solver is a bound gap, and a selector cannot select a strong cut that does not exist. If a learned component is to help this problem, the more promising target is column selection for the position-indexed pricers [34], whose coupled-pair oracle is the measured bottleneck, rather than cut selection for the Benders master. On the pricing side the corresponding non-learned upgrade is the automatic dual stabilisation of Pessoa et al. [40].

9 Declaration on the use of AI tools

In accordance with institutional guidelines on the use of artificial intelligence in academic work, the following is declared.

What was used, and for what. A generative AI assistant was used throughout this internship: for literature search and summarisation; for drafting and copy-editing the prose and the LaTeX source of this report; for scaffolding, debugging and refactoring the experimental code; for producing figures and tables from data; and for developing the statements, proofs and constructions of Section 4.

What was not. The research directions were set by the advisors, and Section 1.3 records which of them proposed which. Running the campaigns on the cluster, diagnosing the missing coverage row, the PORTA enumeration, the instance generator and the cross-solver checking were the author's work.

How the results were checked. Every quantitative claim here is reproduced by a separate verification program, and the numbers in Section 6 are recomputed from the raw per-instance result files rather than carried over from an earlier campaign. Appendix B describes both, says what they cover, and says where their coverage stops.

Responsibility. The author read every passage, line of code and result in this report, and is responsible for its content.

A Proofs

Proof of Theorem 1. Both inequalities are explicit constructions. By Proposition 1 an optimal schedule may be assumed to load by KTNS, so every magazine is full and $|M_i| = b$.

The inequality $Z^* \le \min_{\mathcal{P}} \gamma(\mathcal{P})$. Let $\mathcal{P} = \{G_1, \dots, G_p\}$ be a feasible grouping with configurations $C_1, \dots, C_p$, and let σ attain the minimum in Definition 7. Process the groups in the order $G_{\sigma(1)}, \dots, G_{\sigma(p)}$, and within each group its jobs in any order, holding the magazine at $C_{\sigma(l)}$ throughout group $G_{\sigma(l)}$. This is feasible: each $j \in G_{\sigma(l)}$ satisfies $T_j \subseteq \bigcup_{i \in G_{\sigma(l)}} T_i \subseteq C_{\sigma(l)}$, and $|C_{\sigma(l)}| = b$. The magazine is constant within a group and changes only at the $p-1$ group boundaries, so the free-initial switch cost is $\sum_{l=1}^{p-1} d(C_{\sigma(l)}, C_{\sigma(l+1)}) = \gamma(\mathcal{P})$. Hence $Z^* \le \gamma(\mathcal{P})$, and minimising over $\mathcal{P}$ gives the claim.

The inequality $\min_{\mathcal{P}} \gamma(\mathcal{P}) \le Z^*$. Let $(\pi, \mathbf{M})$ be an optimal schedule with full magazines. Partition the positions into maximal blocks of consecutive positions sharing one magazine. Let these be $B_1, \dots, B_r$ in order, with magazines $D_1, \dots, D_r$, where $D_l \ne D_{l+1}$ by maximality. Each block is a set of jobs run under D_l, so $\bigcup_{j \in B_l} T_j \subseteq D_l$ with $|D_l| = b$; the blocks therefore form a feasible grouping $\mathcal{P}'$ with configurations $D_1, \dots, D_r$. The identity ordering of those configurations is one admissible σ, so $\gamma(\mathcal{P}') \le \sum_{l=1}^{r-1} d(D_l, D_{l+1})$, which is exactly the free-initial switch cost of $(\pi, \mathbf{M})$, namely Z^*. Therefore $\min_{\mathcal{P}} \gamma(\mathcal{P}) \le \gamma(\mathcal{P}') \le Z^*$.

Combining the two gives equality. □

Proof of Proposition 11. The copies use pairwise disjoint tool sets, so $|U| = 6g$ and Proposition 4 gives $Z^* \ge 6g - 3$.

For a schedule attaining this, process the copies one after another, each in the cyclic order of Example 2. Within a copy that loading makes three insertions to build the first magazine and three more across the copy. Since consecutive copies are tool-disjoint, entering a new copy rebuilds its first magazine from scratch, at a cost of three. The total is six insertions for the first copy and six for each of the others, that is $6g$ in the empty-start count and $6g - 3$ free-initial. Hence $Z^* = 6g - 3$.

For the walk value, a minimum-cardinality grouping of R_g uses $K^* = 3g$ configurations, three per copy. Within a copy those three configurations are pairwise at distance two, so any ordering of one copy's three contributes $2 + 2 = 4$. Any transition between configurations of different copies costs three, the copies being tool-disjoint. Connecting g copies requires at least $g - 1$ inter-copy transitions, so every ordering costs at least $4g + 3(g - 1) = 7g - 3$, and that value is attained by keeping each copy's configurations consecutive. Thus $H_{\text{walk}} = 7g - 3$ and $H_{\text{walk}} - Z^* = g$. □

Proof of the walk lemma used in Corollaries 4 and 5, and of Proposition 7(iv). Suppose $Z^* = q$ and take an optimal schedule. Its walk of distinct configurations $D_1, \dots, D_p$ may be assumed, after excising configurations that serve no job, to have every configuration

serve a job: the transition cost obeys the triangle inequality $d(A,C) \le d(A,B) + d(B,C)$, and no covering walk costs less than q. With zero re-insertions every insertion is a first-time insertion, so the $p-1$ transitions insert q tools in total and $p \le q+1$. Assigning each job to a configuration serving it exhibits a feasible p-grouping, so $p \ge K^*$. If $p = K^*$ that grouping is of minimum cardinality and has cost Z^*, forcing $H_{\text{walk}} = Z^*$. In particular $Z^* = q \le 2$ gives $p \le q+1 \le 3 = K^*$ and the gap vanishes, which is the claim used in Proposition 7(i).

For part (iv), let $q = 3$. If $p = 3$ the gap vanishes as above, so let $p = 4$. Each transition then inserts exactly one fresh tool, say $D_{l+1} \setminus D_l = \{x_{l+1}\}$ and $D_l \setminus D_{l+1} = \{e_{l+1}\}$, and zero re-insertion forces *persistence*: a tool lying in $D_i \cap D_j$ with $i < j$ belongs to every intermediate D_l. Write U_l for the requirement of group G_l and suppose $|U_i \cup U_j| \le b$ for some $i < j$. Merging G_i and G_j and keeping the other two groups with their walk configurations yields a minimum-cardinality grouping. It remains to choose its configuration $C \supseteq U_i \cup U_j$ and an ordering whose wrap term $|(\text{first} \cap \text{last}) \setminus \text{middle}|$ of Proposition 6 is at most one, which gives $H_{\text{walk}} \le q + 1 = Z^* + 1$.

For the pair $(1,2)$: take $C \subseteq D_1 \cup D_2$, possible since that union has size $b+1$, and order (C, D_3, D_4). The wrap lies in $(D_1 \cap D_4 \setminus D_3) \cup (D_2 \cap D_4 \setminus D_3) \subseteq \varnothing \cup (D_2 \setminus D_3) = \{e_3\}$, using persistence. For $(3,4)$: order (D_1, D_2, C) with $C \subseteq D_3 \cup D_4$; the wrap lies in $D_1 \cap (D_3 \cup D_4) \setminus D_2 = \varnothing$ by persistence. For $(1,3)$, $(2,4)$ and $(1,4)$: the orderings (C, D_2, D_4), (D_1, D_3, C) and (C, D_2, D_3) bound the wrap by $\{x_3\}$, $\{e_3\}$ and $\{x_3\}$ respectively, by the same two facts. For $(2,3)$: the set $W = U_2 \cup U_3 \cup (D_1 \cap D_4)$ lies in $D_2 \cup \{x_3\}$, because persistence places $D_1 \cap D_4$ inside $D_2 \cap D_3$, so $|W| \le b+1$ and a configuration $C \supseteq U_2 \cup U_3$ inside W omits at most one element of $D_1 \cap D_4$; the ordering (D_1, C, D_4) then has wrap $|(D_1 \cap D_4) \setminus C| \le 1$. □

Proof of Proposition 20. Write $a_t^k = \sum_{C \ni t} y_C^k$ for the presence of tool t at position k, and recall that every configuration holds exactly b tools. If $|U| \le b$ then $q = 0$ and the second claim is immediate, so assume $|U| > b$.

Exactness. In an integer point, (P) places exactly one configuration C_k at each position, and (Cov) gives every job j a position whose configuration contains T_j. Assigning each job to such a position and running the jobs of a position consecutively produces a schedule; no insertion occurs between two jobs at the same position, so its free-initial cost is $\sum_{k \ge 2} d(C_{k-1}, C_k)$. Minimising a non-negative objective drives (W) to equality, so that sum is exactly the objective. Conversely a schedule with magazines $C_1, \dots, C_n$ gives an integer point of the same cost. The two sets therefore correspond with equal cost.

Without (T) the relaxation is 0. Every job satisfies $|T_j| \le b$, so each T_j extends to at least one configuration; choose one per job and let $\mathcal{C}$ be the resulting set, of size $m \le n$. Put $y_C^k = 1/m$ for every $C \in \mathcal{C}$ and every k. Then (P) holds with equality, and for each

job j,

$$\sum_k \sum_{C \supseteq T_j} y_C^k \;=\; n \cdot \frac{|\{C \in \mathcal{C} : C \supseteq T_j\}|}{m} \;\geq\; \frac{n}{m} \;\geq\; 1,$$

so (Cov) holds. The point does not depend on k, so the right-hand side of (W) is 0 at every position and $w \equiv 0$ is feasible. The objective is 0.

With (T) the relaxation is at least q. Summing (T) over $t \in U$,

$$\sum_{t \in U} \sum_{k \geq 2} w_t^k \;\geq\; |U| - \sum_{t \in U} a_t^1 \;\geq\; |U| - \sum_{t \in T} a_t^1 \;=\; |U| - b \sum_C y_C^1 \;=\; |U| - b \;=\; q,$$

using $|C| = b$ and (P) at $k = 1$. The left-hand side is at most the objective.

And at most q. Since $|U| > b$, the configurations of $\mathcal{C}$ may be chosen inside U, so $|C \cap U| = b$ for each. Take the blended point above and set $w_t^2 = 1 - a_t^1$ for $t \in U$ and $w_t^k = 0$ otherwise. Each value lies in $[0, 1]$, and (W) at $k = 2$ has right-hand side 0, so the point is feasible; (T) holds with equality. Its objective is

$$\sum_{t \in U} \left(1 - a_t^1\right) \;=\; |U| - \sum_C y_C^1 |C \cap U| \;=\; |U| - b \;=\; q. \qquad \square$$

Proof of Proposition 22. Exactness follows the argument for PCF, with a column now carrying an ordered pair of consecutive configurations and the chaining rows forcing the second element of the pair at position k to agree, tool by tool, with the first element at position $k + 1$.

The relaxation is at least q. PTF carries the same per-tool counting rows as PCF, and the summation in the previous proof uses only those rows together with $|C| = b$ at the first position. Both hold in PTF, so the same chain of inequalities applies unchanged.

It can be strictly greater, and this half is established by exhibiting witnesses rather than by a general argument. Three appear in the campaign of Section 6.6, where the root relaxation is recorded on every run: on Catanzaro `A0-0` it is 2.0134 against $q = 2$, and on Laporte3 `L11-6` and `L11-7` it is 6 against $q = 5$. A witness is all the statement requires; how large the excess can be made is Problem 2 and is open. $\square$

Proof of Proposition 21. Exactness. Take an optimal schedule and collapse equal consecutive magazines to the sequence of distinct magazines it visits, a trajectory $D_1, \ldots, D_m$ with $m \leq n$, as in the proof of Theorem 1. Set $y_{D_l}^l = 1$ for $l = 1, \ldots, m$, leave positions $m + 1, \ldots, n$ empty, and put $w_t^k = \max(0, a_t^k - a_t^{k-1})$. Then (P′) holds, with value 1 on the prefix and 0 afterwards, and (G) holds because the prefix is contiguous, while (Cov), (W) and (T) hold exactly as for PCF. The step from position m to position $m + 1$ and all later steps add no cost, so the objective is $\sum_{l<m} d(D_l, D_{l+1}) = Z^*$. Hence the optimum is at most Z^*.

Conversely, any integer-feasible PCF′ point has, by (G), its used positions in a

prefix $1, \ldots, p$, each holding one configuration D_k. By (W) the objective is at least $\sum_{k=2}^{p} d(D_{k-1}, D_k)$, and by (Cov) every job is served by some D_k, so assigning each job to a covering position yields a schedule of cost at most the objective. Hence the optimum is at least Z^*.

Invariance of the relaxation. Every PCF-feasible point has $\sum_C y_C^k = 1$ for all k and therefore satisfies (P′) and (G). The PCF′ relaxation minimises over a superset and so is no larger than PCF's. For the reverse, sum (T) over $t \in U$ and use $\sum_{t \in U} a_t^1 \leq \sum_{t \in T} a_t^1 = b \sum_C y_C^1 \leq b$, where the last bound now comes from (P′) rather than (P). This gives objective at least $|U| - b$ at every PCF′ point, so the two relaxations coincide wherever PCF's value is q. Dropping (T), the plain relaxation is certified to be 0 by the same blended configuration used for PCF. □

B How the results were checked

This appendix says how the results of this report were checked, so that a reader who wants to repeat the check can.

B.1 How the program works

It shares no code with the solvers, the campaign harness, or any script written during the research, so where the two agree, that is a second opinion rather than the same calculation run twice. It also avoids using the results it is checking. The loading rule of Proposition 1 is implemented separately and compared against an exact dynamic program over magazine states rather than assumed optimal, and the optimum Z^* comes from a dynamic program over subsets of jobs, cross-checked against brute-force enumeration on the smallest instances, which uses none of the methods of Section 5. On small instances it enumerates rather than samples; where a family is too large to enumerate, the sample and its generation parameters are given and the statement is labelled an Observation rather than a Proposition.

B.2 What is covered

- **The loading oracle.** KTNS against the exact dynamic program, on 349,872 instance-and-sequence pairs, including every permutation of every instance in a random bank. No disagreement. The same comparison was run against the implementation used by the solvers, with the same result.
- **The convention identity** of Proposition 5, on random instances.
- **Both lower bounds** and Corollary 1, on every instance in the random census.

- **Every named instance** in Sections 2 and 4: the k-ring for $k = 3, \ldots, 9$ in both readings; the sliding-window ring; the witness W of Proposition 9; the pair I_0, I_1 of Proposition 14; the $K^* = 4$ instance; the smallest sub-optimal instance of Observation 3; the copies of Observation 5; the star and non-matroid instances of Propositions 12 and 13.
- **Every bound in Section 4.2 and 4.3**, checked for violations on a random census of 200 instances: Theorem 2, Corollaries 3, 4 and 5, Lemma 2, and Propositions 7 and 8. No violation.
- **The closed form of Proposition 6**, on instances with $K^* = 3$ drawn at random until 45 were obtained. No disagreement.
- **The covering relaxation** of Proposition 16, by solving the fractional cover as a linear program for $k = 3, \ldots, 8$. This is what identified that the statement requires $k \geq 4$.
- **The setup-cost collapse** of Theorem 3, by enumerating all feasible groupings and all orderings of an instance and minimising the augmented objective directly. This is also what identified that the threshold must be read in the walk value.
- **The PCF relaxation** of Proposition 20, by building the linear program explicitly over all configurations and solving it, with and without the counting rows.

B.3 What the verification found

Two findings changed the content of this report.

The first is the distinction of Section 2.5. The k-ring family is treated in the literature as the canonical example of a positive gap for the two-phase heuristic, with the gap unbounded on copies of it. The verification found that every k-ring has *zero* gap for the method as specified, and that the published values are walk values. That is why Section 2.5 exists, why Table 2 has two gap columns, and why Proposition 11 is now stated for the walk value with Observation 5 supplying a different family for the other reading.

The second is Proposition 16, which had been stated for all k and fails at $k = 3$, where the whole job set forms a single feasible group.

Both were errors of statement rather than of proof: the arguments were sound for the objects they were actually about.

A third finding concerns the campaign analysis rather than the structural results. Two properties of the raw result files can silently corrupt any analysis of them; both are in the code that reads the results rather than in any solver, and Appendix C states them.

B.4 Where the checking stops

The checking is exhaustive only on small instances. On larger ones it compares methods against each other, which catches disagreement but not an error they share. And reproducing a number confirms the number rather than the argument given for it, so the proofs were re-derived by hand as well.

B.5 Running it

The program is `verification/verify_report_independent.py` in the repository. It requires Python 3 and SciPy, needs no commercial solver, and completes in roughly twenty-five minutes. It prints one line per claim, naming the section of this report the claim comes from, the value that section states, and the value the computation produces, and exits with a non-zero status if any of them disagree.

The campaign numbers of Section 6 are produced by a separate analysis over the raw per-instance result files, described in Appendix C. No number in Section 6 is carried over from an earlier campaign.

C Reproducing the experiments

The complete implementation — the solvers, the reimplemented baselines, the campaign harness, the verification program, and the per-instance result files behind every number in Section 6 — is available at `https://github.com/shankar-n/JGP-SSP-Codes-Shankar`.

C.1 Layout

Path	Contents
`src/SSP/`	the loading oracle, instance handling, heuristics, the grouping solver
`src/BBC/`	the branch-and-Benders-cut solver, the three reimplemented baselines, and the campaign harness
`src/BNP/`	the PCF′ and PTF branch-and-price prototypes and their pricers
`data/`	the instance families of Table 5
`verification/`	the independent verification program of Appendix B
`report/`	the source of this document

C.2 Requirements

The exact solvers require IBM CPLEX with its Python interface; version 22.1.1 was used throughout. The branch-and-price prototypes require PySCIPOpt. The grouping solver requires PySCIPOpt. The verification program of Appendix B requires only Python 3 and SciPy, and in particular needs no commercial solver, so the structural results of Section 4 can be checked on any machine.

C.3 Instance format

An instance file gives the number of jobs, the number of tools and the capacity on one line, followed by the job–tool incidence matrix with one row per tool and one column per job, where entry (t, j) is 1 when job j requires tool t.

C.4 Running the campaign

The instance families and the solver configurations are declared in one place, in the campaign configuration module, which is the single source of truth for both. The runner is resumable: it records one row per completed run and skips runs already recorded. That property requires care. If the solver changes, previously recorded rows must be deleted before re-running, or the runner will keep the stale results. Both re-runs described in this report — the one following the Benders cut correction and the one following the addition of the coverage row — required exactly that.

On the cluster the campaign is submitted as a SLURM job array with one task per configuration and instance shard. Each run is given four dedicated threads, 16 gigabytes of memory and a one-hour limit.

C.5 Reproducing the numbers in Section 6

Every table in Section 6 is computed from the raw per-instance result files by a single analysis pass, which reads the shards, keys each row by the five-part identifier (family, file name, jobs, tools, capacity), and derives:

- the solved counts of Table 6, per family and in total, against the number of runs that returned a result;
- the cross-method agreement check, comparing the empty-start cost of the returned sequence across every pair of methods that both solved an instance;
- the shifted geometric mean times of Table 7, on the subset that all compared configurations solve;
- the fractional-cut comparison of Table 9 and the ablation of Table 10, each on the instances that both compared configurations attempted;

- the coverage-bound split of Table 11, by reading each instance file, counting the tools $|U|$ that some job actually requires, and testing whether the known optimum equals $|U|$ in the empty-start cost;
- the branch-and-price table and root-bound counts of Section 6.6, from the separate result files of the prototypes.

Two points of the key matter, because getting either wrong changes the conclusions. The capacity must be part of the identifier: the Crama collection reuses file names across four capacities, and keying on the name alone merges rows from different instances and manufactures cross-method disagreements that do not exist. And the coverage bound must be computed from $|U|$ rather than from the tool count declared in the instance file, because ten instances of the collection declare a tool that no job requires.

Comparisons are made on common subsets rather than on absolute counts wherever the denominators differ, which is what makes the conclusions of Section 7 insensitive to the runs lost as described in Section 6.2.

References

[1] Najmaddin Akhundov and James Ostrowski. Exploiting symmetry for the job sequencing and tool switching problem. *European Journal of Operational Research*, 316(3):976–987, 2024.

[2] David L. Applegate, Robert E. Bixby, Vašek Chvátal, and William J. Cook. *The Traveling Salesman Problem: A Computational Study*. Princeton University Press, 2006.

[3] Alper Atamtürk, George L. Nemhauser, and Martin W. P. Savelsbergh. Conflict graphs in solving integer programming problems. *European Journal of Operational Research*, 121(1):40–55, 2000.

[4] Cynthia Barnhart, Ellis L. Johnson, George L. Nemhauser, Martin W. P. Savelsbergh, and Pamela H. Vance. Branch-and-price: Column generation for solving huge integer programs. *Operations Research*, 46(3):316–329, 1998.

[5] László A. Belady. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, 5(2):78–101, 1966.

[6] Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. Machine learning for combinatorial optimization: a methodological tour d'horizon. *European Journal of Operational Research*, 290(2):405–421, 2021.

[7] Alewyn P. Burger, C. G. Jacobs, Jan H. van Vuuren, and Stephan E. Visagie. Scheduling multi-colour print jobs with sequence-dependent setup times. *Journal of Scheduling*, 18(2):131–145, 2015.

[8] Dorothea Calmels. The job sequencing and tool switching problem: state-of-the-art literature review, classification, and trends. *International Journal of Production Research*, 2018. Published online 6 August 2018; version of record *Int. J. Prod. Res.* **57**(15–16), 5052–5068, 2019.

[9] Daniele Catanzaro, Luis Gouveia, and Martine Labbé. Improved integer linear programming formulations for the job sequencing and tool switching problem. *European Journal of Operational Research*, 244(3):766–777, 2015.

[10] Maria Chudnovsky, Neil Robertson, Paul Seymour, and Robin Thomas. The strong perfect graph theorem. *Annals of Mathematics*, 164(1):51–229, 2006.

[11] Gianni Codato and Matteo Fischetti. Combinatorial Benders' cuts for mixed-integer linear programming. *Operations Research*, 54(4):756–766, 2006.

[12] Rafael Colares and Annegret Wagler. Exact formulations for the job sequencing and tool switching problem. Working paper, Université Clermont Auvergne, LIMOS, 2026.

[13] Yves Crama and A. G. Oerlemans. A column generation approach to job grouping for flexible manufacturing systems. *European Journal of Operational Research*, 78(1): 58–80, 1994.

[14] Yves Crama, Antoon W. J. Kolen, Alwin G. Oerlemans, and Frits C. R. Spieksma. Minimizing the number of tool switches on a flexible machine. *International Journal of Flexible Manufacturing Systems*, 6(1):33–54, 1994.

[15] Yves Crama, Linda S. Moonen, Frits C. R. Spieksma, and Ellen Talloen. The tool switching problem revisited. *European Journal of Operational Research*, 182(2): 952–957, 2007.

[16] Tiago Tiburcio da Silva, Antônio Augusto Chaves, and Horacio Hideki Yanasse. A new multicommodity flow model for the job sequencing and tool switching problem. *International Journal of Production Research*, 59(12):3617–3632, 2021.

[17] Martin Desrochers and Gilbert Laporte. Improvements and extensions to the Miller–Tucker–Zemlin subtour elimination constraints. *Operations Research Letters*, 10(1): 27–36, 1991.

[18] Arnaud Deza and Elias B. Khalil. Machine learning for cutting planes in integer programming: A survey. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence (IJCAI)*, 2023.

[19] Matteo Fischetti, Juan José Salazar-González, and Paolo Toth. A branch-and-cut algorithm for the symmetric generalized traveling salesman problem. *Operations Research*, 45(3):378–394, 1997.

[20] Maxime Gasse, Didier Chételat, Nicola Ferroni, Laurent Charlin, and Andrea Lodi. Exact combinatorial optimization with graph convolutional neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.

[21] Gianpaolo Ghiani, Antonio Grieco, and Emanuela Guerriero. Solving the job sequencing and tool switching problem as a nonlinear least cost hamiltonian cycle problem. *Networks*, 55(4):379–385, 2010.

[22] Olivier Goldschmidt, David Nehme, and Gang Yu. Note on the set-union knapsack problem. *Naval Research Logistics*, 41(6):833–842, 1994.

[23] Keld Helsgaun. Solving the equality generalized traveling salesman problem using the lin–kernighan–helsgaun algorithm. *Mathematical Programming Computation*, 7 (3):269–287, 2015.

[24] Alain Hertz, Gilbert Laporte, Michel Mittaz, and Kathryn E. Stecke. Heuristics for minimizing tool switches when scheduling part types on a flexible machine. *IIE Transactions*, 30(8):689–694, 1998.

[25] John N. Hooker and Greger Ottosson. Logic-based benders decomposition. *Mathematical Programming*, 96(1):33–60, 2003.

[26] Roel Jans and Jacques Desrosiers. Efficient symmetry breaking formulations for the job grouping problem. *Computers & Operations Research*, 40(4):1132–1142, 2013.

[27] Layth Kashou, André Rossi, Evgeny Gurevsky, and Rui S. Shibasaki. Benders decomposition for robust balancing of production lines. M2 internship report, Université Paris-Saclay (LAMSADE, Université Paris-Dauphine–PSL), 2025.

[28] Gilbert Laporte, Juan José Salazar-González, and Frédéric Semet. Exact algorithms for the job sequencing and tool switching problem. *IIE Transactions*, 36(1):37–45, 2004.

[29] Emma Legrand, Vianney Coppé, Daniele Catanzaro, and Pierre Schaus. A dynamic programming approach for the job sequencing and tool switching problem. In *Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR 2025)*, volume 15763 of *Lecture Notes in Computer Science*, pages 70–85. Springer, 2025.

[30] Marco E. Lübbecke and Jacques Desrosiers. Selected topics in column generation. *Operations Research*, 53(6):1007–1023, 2005.

[31] Thomas L. Magnanti and Richard T. Wong. Accelerating Benders decomposition: Algorithmic enhancement and model selection criteria. *Operations Research*, 29(3): 464–484, 1981.

[32] Jordana Mecler, Anand Subramanian, and Thibaut Vidal. A simple and effective hybrid genetic search for the job sequencing and tool switching problem. *Computers & Operations Research*, 127:105153, 2021.

[33] Clair E. Miller, Albert W. Tucker, and Richard A. Zemlin. Integer programming formulation of traveling salesman problems. *Journal of the ACM*, 7(4):326–329, 1960.

[34] Mouad Morabit, Guy Desaulniers, and Andrea Lodi. Machine-learning-based column selection for column generation. *Transportation Science*, 55(4):815–831, 2021.

[35] Bernardo Bastos Pereira Moreira. Approaches based on magazine configuration to solve the job sequencing and tool switching problem. Master's thesis, Universidade Federal de Minas Gerais (UFMG), Belo Horizonte, Brazil, 2026.

[36] İbrahim Muter, Ş. İlker Birbil, and Kerem Bülbül. Simultaneous column-and-row generation for large-scale linear programs with column-dependent-rows. *Mathematical Programming*, 142(1–2):47–82, 2013.

[37] Charles E. Noon and James C. Bean. An efficient transformation of the generalized traveling salesman problem. *INFOR: Information Systems and Operational Research*, 31(1):39–44, 1993.

[38] Felipe Shoei Otiai. Efficient tool management in flexible manufacturing: A branch and price approach to the job grouping problem. Master's thesis, ISIMA, Clermont Auvergne INP, 2024.

[39] Nikolaos Papadakos. Practical enhancements to the Magnanti–Wong method. *Operations Research Letters*, 36(4):444–449, 2008.

[40] Artur Pessoa, Ruslan Sadykov, Eduardo Uchoa, and François Vanderbeck. Automation and combination of linear-programming based stabilization techniques in column generation. *INFORMS Journal on Computing*, 30(2):339–360, 2018.

[41] Marcus Poggi and Eduardo Uchoa. Chapter 3: New exact algorithms for the capacitated vehicle routing problem. In *Vehicle Routing: Problems, Methods, and Applications*, pages 59–86. SIAM, 2014. Source for the robust/non-robust cut distinction used in Section 7.2.

[42] Petrică C. Pop, Ovidiu Cosma, Cosmin Sabo, and Corina Pop Sitar. The generalized traveling salesman problem: A comprehensive review. *European Journal of Operational Research*, 314(3):819–835, 2024.

[43] Ragheb Rahmaniani, Teodor Gabriel Crainic, Michel Gendreau, and Walter Rei. The Benders decomposition algorithm: A literature review. *European Journal of Operational Research*, 259(3):801–817, 2017.

[44] D. M. Ryan and B. A. Foster. An integer programming approach to scheduling. *Computer Scheduling of Public Transport*, pages 269–280, 1981.

[45] Christopher S. Tang and Eric V. Denardo. Models arising from a flexible manufacturing machine, part i: Minimization of the number of tool switches. *Operations Research*, 36(5):767–777, 1988.

[46] Yunhao Tang, Shipra Agrawal, and Yuri Faenza. Reinforcement learning for integer programming: Learning to cut. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119 of *PMLR*, pages 9367–9376, 2020.

[47] Horacio Hideki Yanasse, Ricardo de Carvalho Miranda Rodrigues, and Edson Luiz França Senne. An enumerative algorithm based on partial ordering for the minimization of tool switches problem. *Gestão & Produção*, 16(3):303–314, 2009.